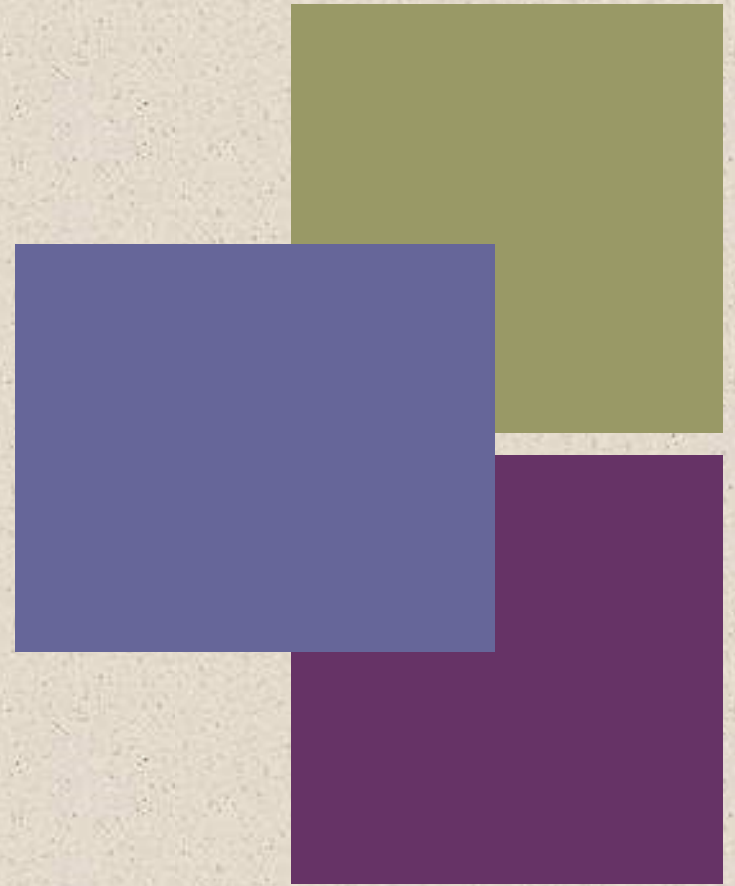


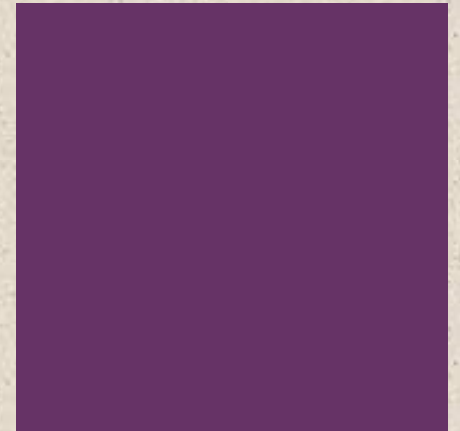
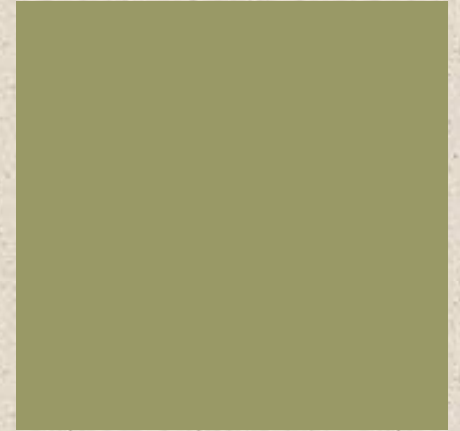
+ Chapter 11

Digital Logic



Questions-Prelude

1. Define a program.
2. How is a program stored and executed?
3. Who executes a program?
4. Where is the program stored and how do you estimate the size of a program?
5. How to locate an instruction in a memory?
6. What do you mean by “addressing capacity” of a computer system?



Class-1-Questions

1. Define a signal.
2. What are the types of signals?
3. Differentiate among all of them.
4. What is the need for digital signals?
5. Define Boolean Algebra.
6. Who Proposed it?
7. Why it is used?

+



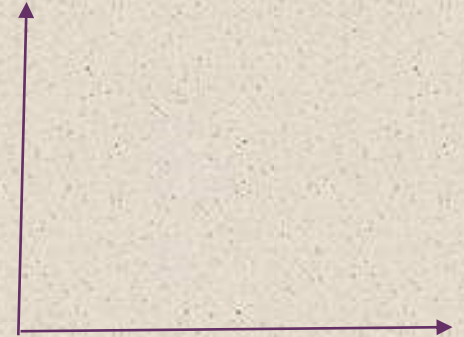
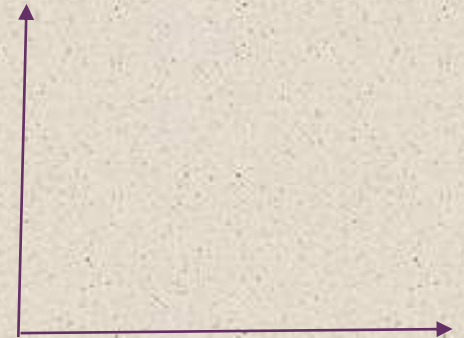
Signal Definition

- A **signal** is a function that represents the variation of a physical quantity with respect to any parameter.
- Generally Parameter is Time/Distance
- **In Electrical or electronics**
- Physical Quantity: Current/Voltage
- Parameter: Time



Analog Signal

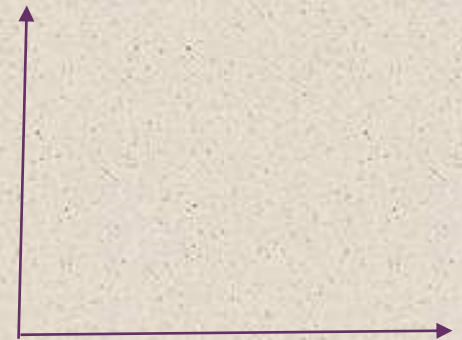
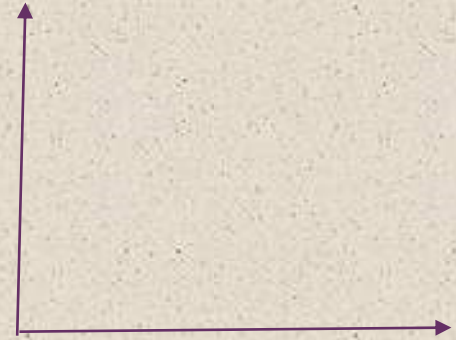
- The signals that can take any value between two limits
- It is continuous in nature
- Virtually all naturally occurring physical phenomena are analog signals.
- Examples: Temperature, pressure, velocity, and sound, for instance, are signals that take on all possible values between given limits





Discrete Time Signal

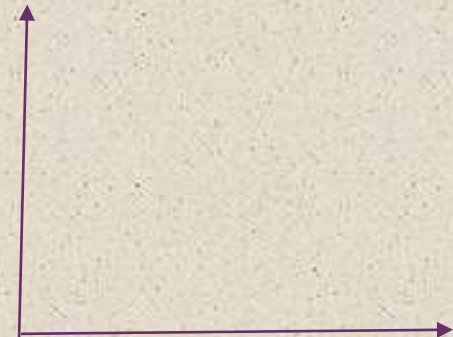
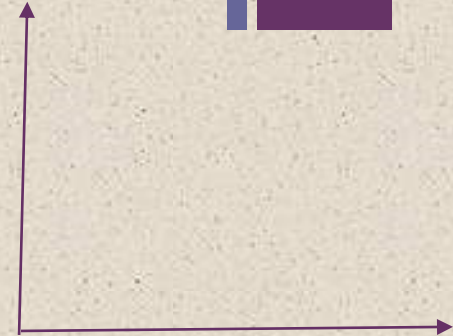
- The signal that is defined for discrete intervals of time
- It is a subset of Analog signals





Digital Signal

- In digital signals both time and magnitude are discretized.
- Discretize the time axis into equal intervals
- Discretize the magnitude axis into a fixed no of levels.
- Select the lower level to minimize the error.
- Error will reduced if no of levels are increased.



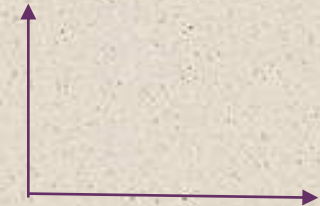


Need of Digital Signal

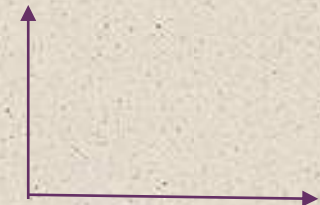
- Minimizing the noise: Unwanted signal



Transmitter



Receiver





Boolean Algebra



- Mathematical discipline used to design and analyze the behavior of the digital circuitry in digital computers and other digital systems
- Named after George Boole an English mathematician who proposed basic principles of this algebra in 1854.
- Is a convenient tool:
 - **Analysis**
 - It is an economical way of describing the function of digital circuitry
 - **Design**
 - Given a desired function, Boolean algebra can be applied to develop a simplified implementation of that function



Class-2-Questions

- What are operations carried out in Boolean Algebra?





Boolean Variables and Operations



- Makes use of variables and operations
 - Are logical
 - A variable may take on the value 1 (TRUE) or 0 (FALSE)
 - Basic logical operations are AND, OR, and NOT
- AND
 - Yields true (binary value 1) if and only if both of its operands are true
 - In the absence of parentheses the AND operation takes precedence over the OR operation
 - When no ambiguity will occur the AND operation is represented by simple concatenation instead of the dot operator
- OR
 - Yields true if either or both of its operands are true
- NOT
 - Inverts the value of its operand

Table 11.1 Boolean Operators



(a) Boolean Operators of Two Input Variables

P	Q	NOT P ($\bar{P}$)	P AND Q ($P \cdot Q$)	P OR Q ($P + Q$)	P NAND Q ($\overline{P \cdot Q}$)	P NOR Q ($\overline{P + Q}$)	P XOR Q ($P \oplus Q$)
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

(b) Boolean Operators Extended to More than Two Inputs (A, B, ...)

Operation	Expression	Output = 1 if
AND	$A \cdot B \cdot \dots$	All of the set {A, B, ...} are 1.
OR	$A + B + \dots$	Any of the set {A, B, ...} are 1.
NAND	$\overline{A \cdot B \cdot \dots}$	Any of the set {A, B, ...} are 0.
NOR	$\overline{A + B + \dots}$	All of the set {A, B, ...} are 0.
XOR	$A \oplus B \oplus \dots$	The set {A, B, ...} contains an odd number of ones.



Table 11.2

Basic Identities of Boolean Algebra

Basic Postulates

$A \cdot B = B \cdot A$	$A + B = B + A$	Commutative Laws
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Distributive Laws
$1 \cdot A = A$	$0 + A = A$	Identity Elements
$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$	Inverse Elements

Other Identities

$0 \cdot A = 0$	$1 + A = 1$	
$A \cdot A = A$	$A + A = A$	
$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$	Associative Laws
$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$	DeMorgan's Theorem

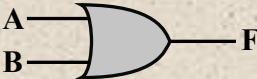
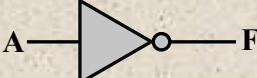
Name	Graphical Symbol	Algebraic Function	Truth Table															
AND		$F = A \bullet B$ or $F = AB$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \overline{A}$ or $F = A'$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
NAND		$F = \overline{AB}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR		$F = A \oplus B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Figure 11.1 Basic Logic Gates

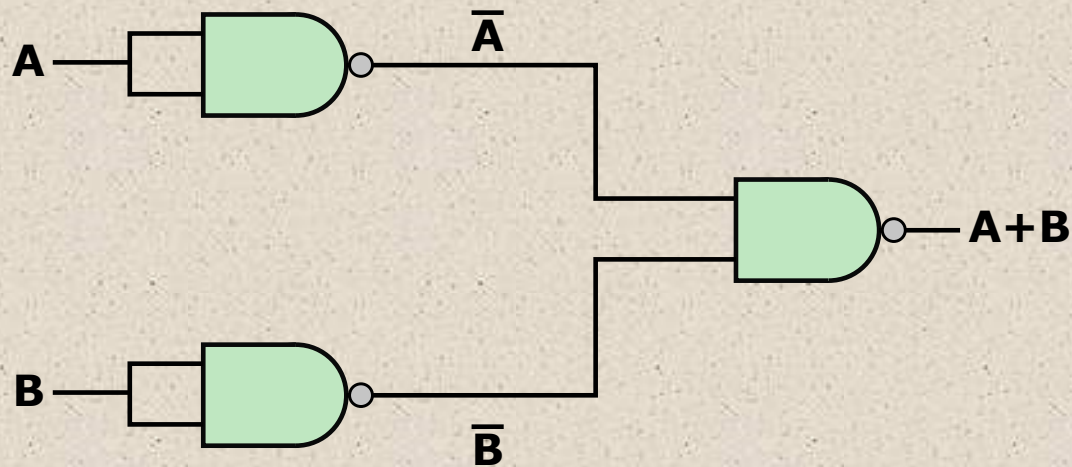
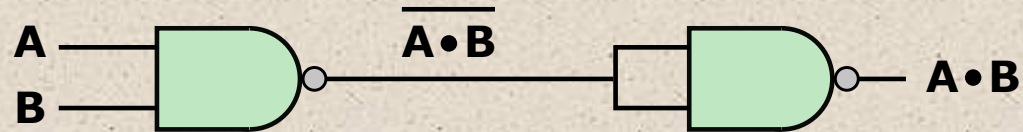
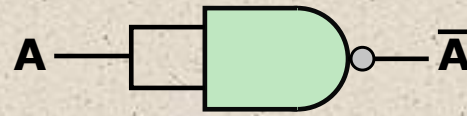


Figure 11.2 Some Uses of NAND Gates

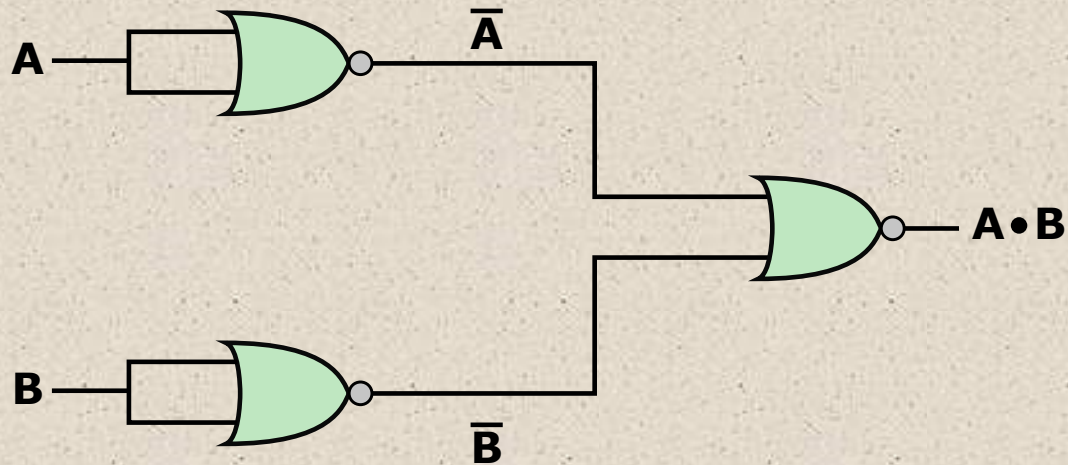
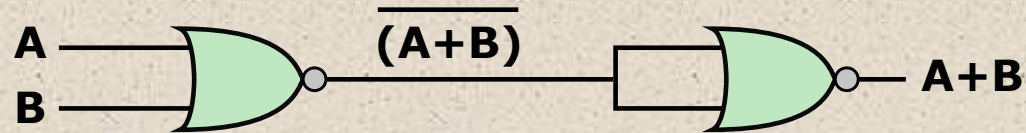
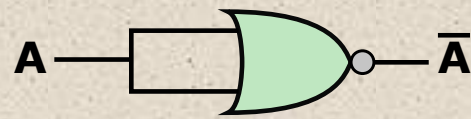
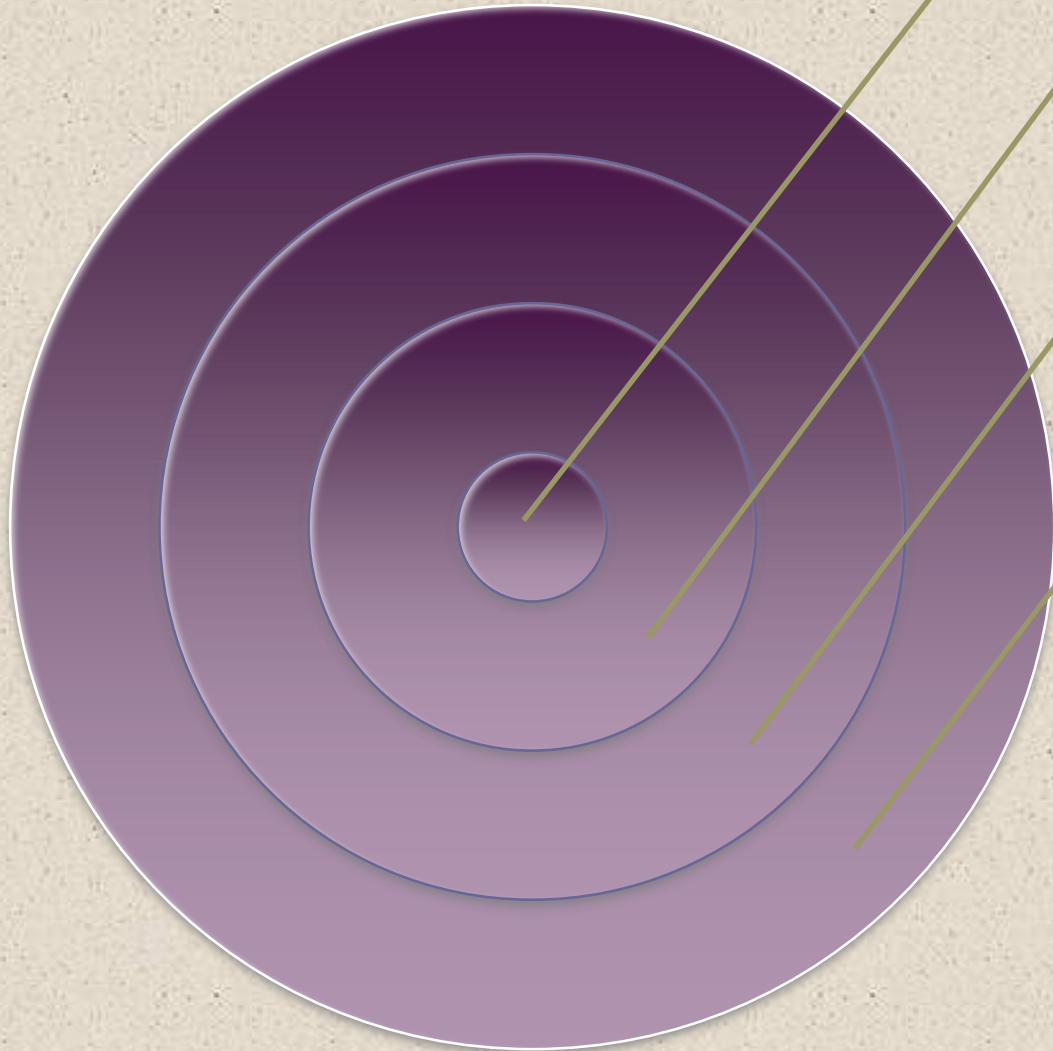


Figure 11.3 Some Uses of NOR Gates

Combinational Circuit



An interconnected set of gates whose output at any time is a function only of the input at that time

The appearance of the input is followed almost immediately by the appearance of the output, with only gate delays

Consists of n binary inputs and m binary outputs

Can be defined in three ways:

- Truth table
 - For each of the 2^n possible combinations of input signals, the binary value of each of the m output signals is listed
- Graphical symbols
 - The interconnected layout of gates is depicted
- Boolean equations
 - Each output signal is expressed as a Boolean function of its input signals



Table 11.3
A Boolean Function of Three Variables

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

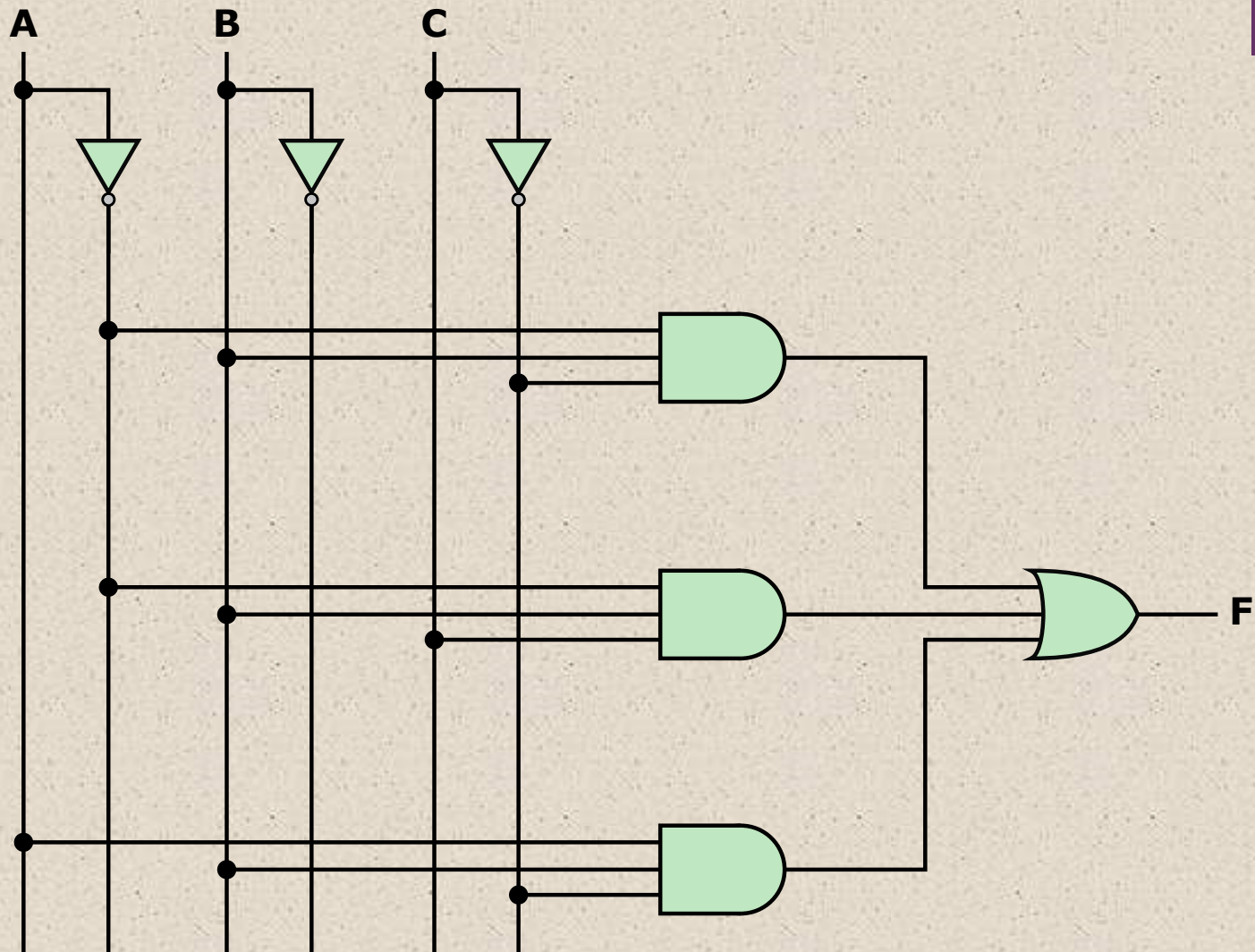


Figure 11.4 Sum-of-Products Implementation of Table 11.3

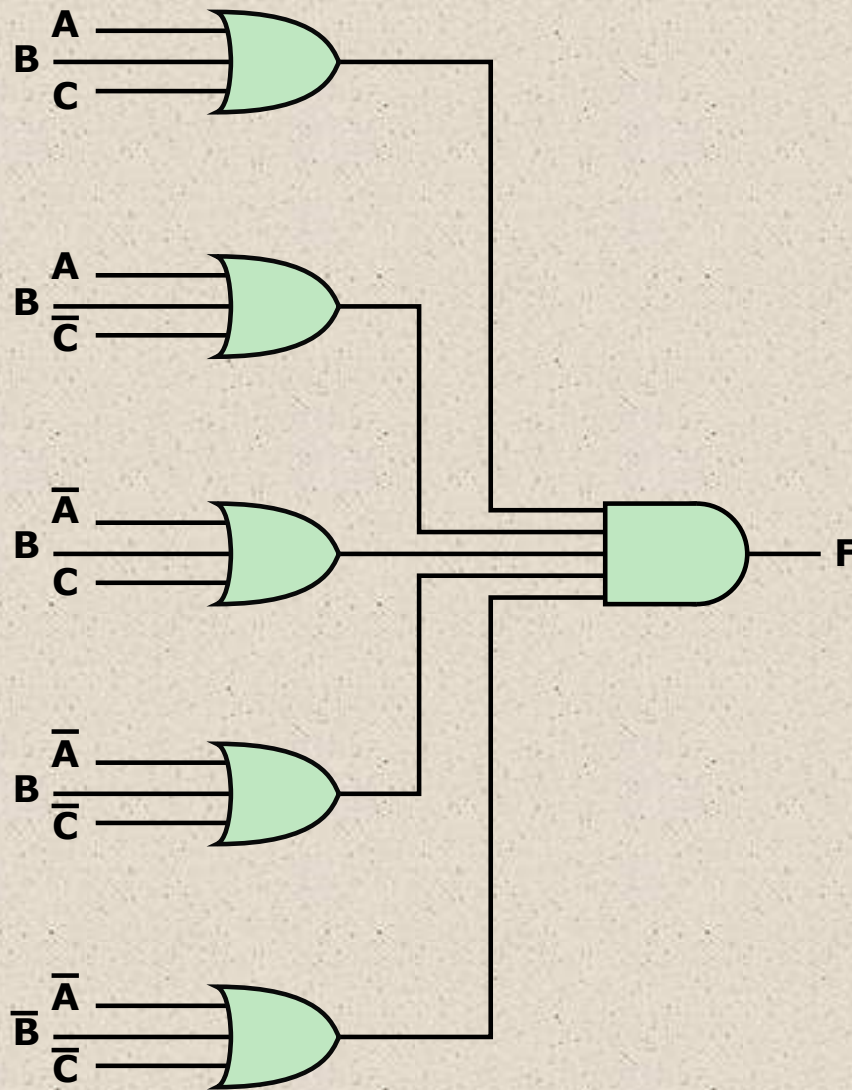


Figure 11.5 Product-of-Sums Implementation of Table 11.3

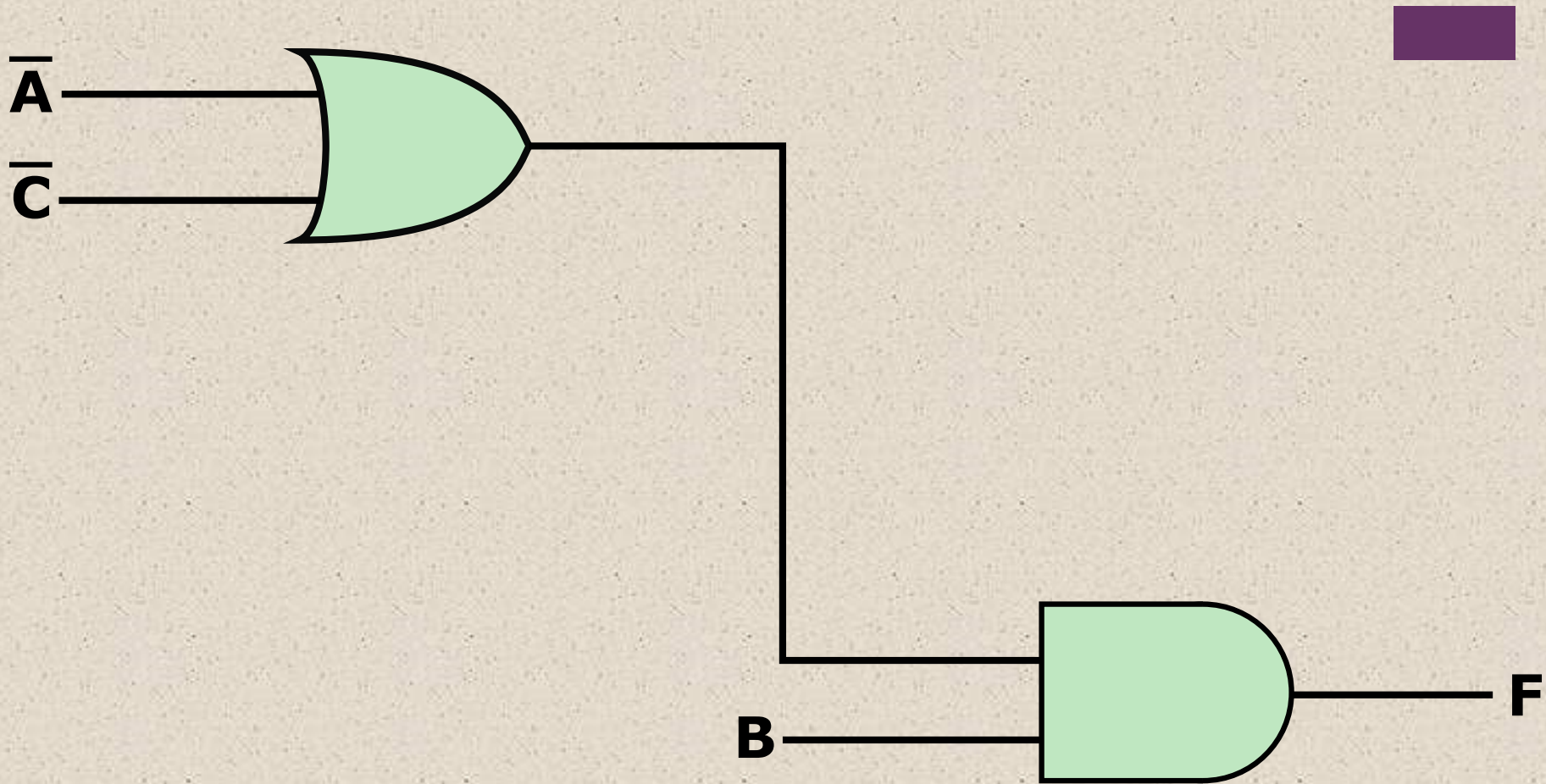


Figure 11.6 Simplified Implementation of Table 11.3

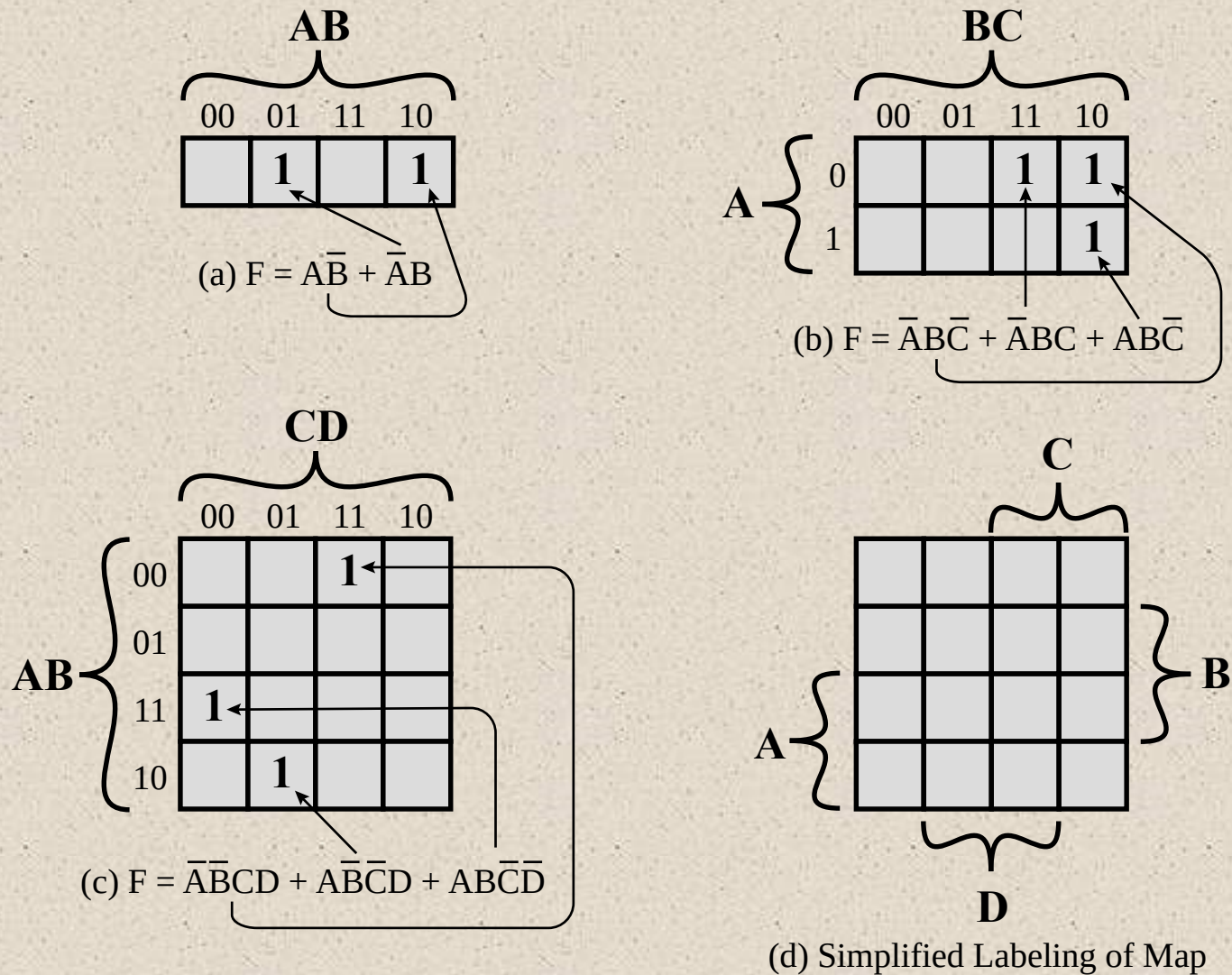
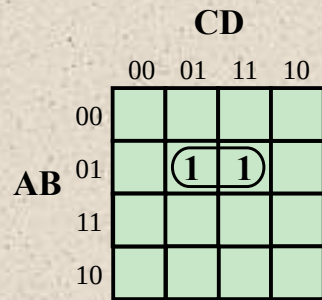
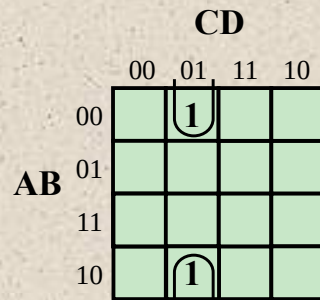


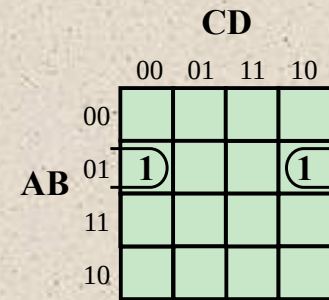
Figure 11.7 The Use of Karnaugh Maps to Represent Boolean Functions



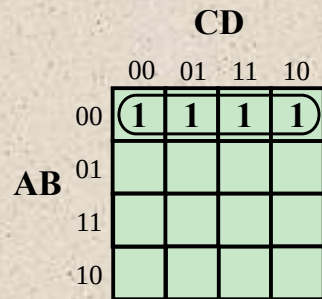
(a) $\bar{A}BD$



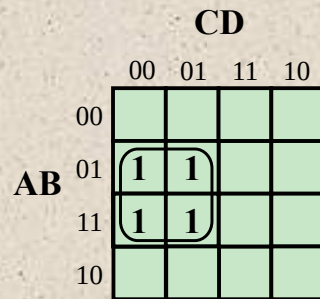
(b) $\bar{B}\bar{C}D$



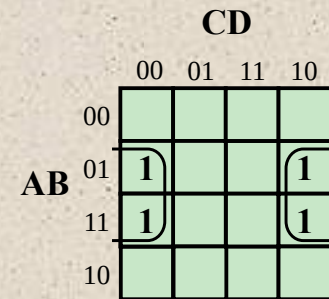
(c) $\bar{A}B\bar{D}$



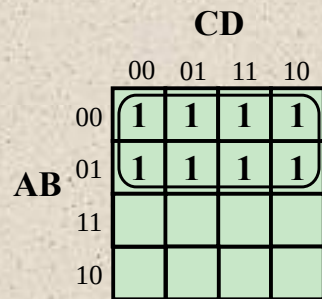
(d) $\bar{A}\bar{B}$



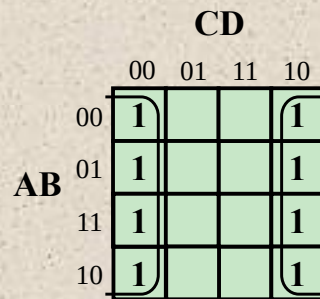
(e) $B\bar{C}$



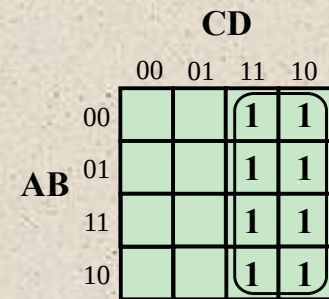
(f) $B\bar{D}$



(g) $\bar{A}$



(h) $\bar{D}$



(i) C

Figure 11.8 Example Use of Karnaugh Maps

BC

		00	01	11	10
A	0			1	1
	1				1

(a) $F = \bar{A}B + B\bar{C}$

CD

		00	01	11	10
	00				
AB	01		1		
	11		1	1	
	10			1	

(b) $F = \bar{B}\bar{C}D + ACD$

Figure 11.9 Overlapping Groups



Table 11.4
Truth Table for the One-Digit Packed
Decimal Incrementer

Input					Output				
Number	A	B	C	D	Number	W	X	Y	Z
0	0	0	0	0	1	0	0	0	1
1	0	0	0	1	2	0	0	1	0
2	0	0	1	0	3	0	0	1	1
3	0	0	1	1	4	0	1	0	0
4	0	1	0	0	5	0	1	0	1
5	0	1	0	1	6	0	1	1	0
6	0	1	1	0	7	0	1	1	1
7	0	1	1	1	8	1	0	0	0
8	1	0	0	0	9	1	0	0	1
9	1	0	0	1	0	0	0	0	0
Don't care con- dition	1	0	1	0		d	d	d	d
	1	0	1	1		d	d	d	d
	1	1	0	0		d	d	d	d
	1	1	0	1		d	d	d	d
	1	1	1	0		d	d	d	d
	1	1	1	1		d	d	d	d

Table 11.4 Truth Table for the One-Digit Packed Decimal Incrementer

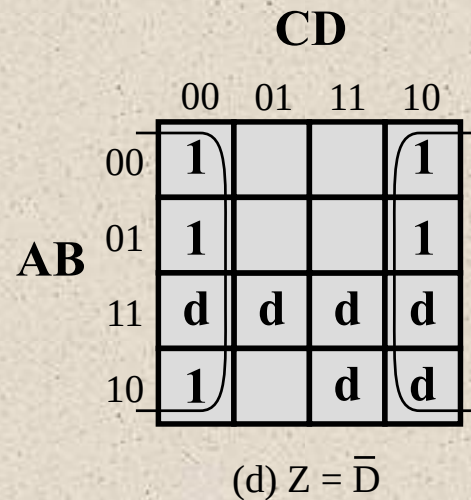
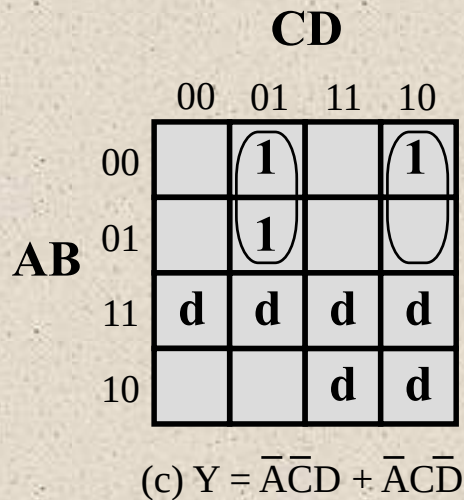
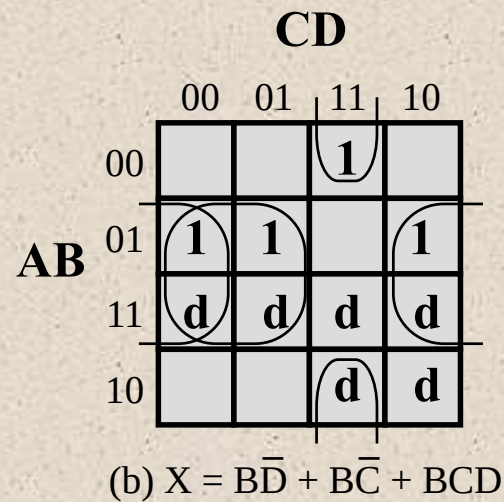
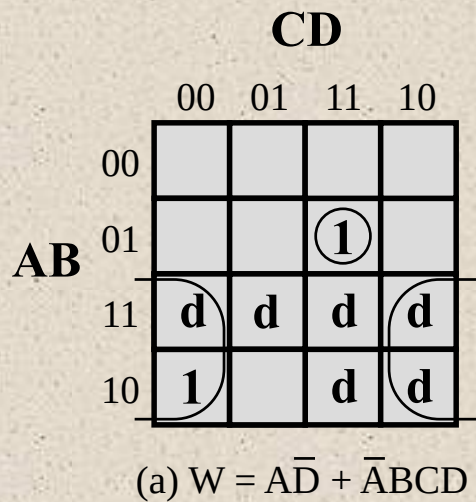


Figure 11.10 Karnaugh Maps for the Incrementer

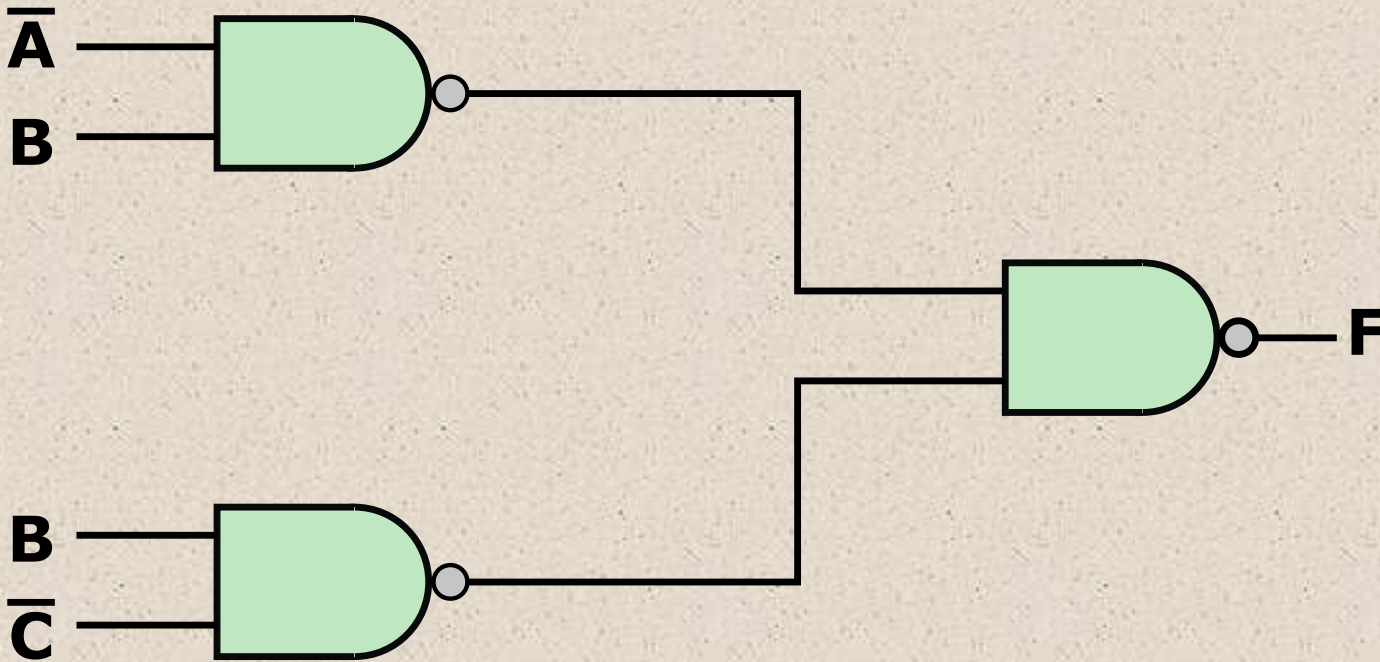


Figure 11.11 NAND Implementation of Table 11.3

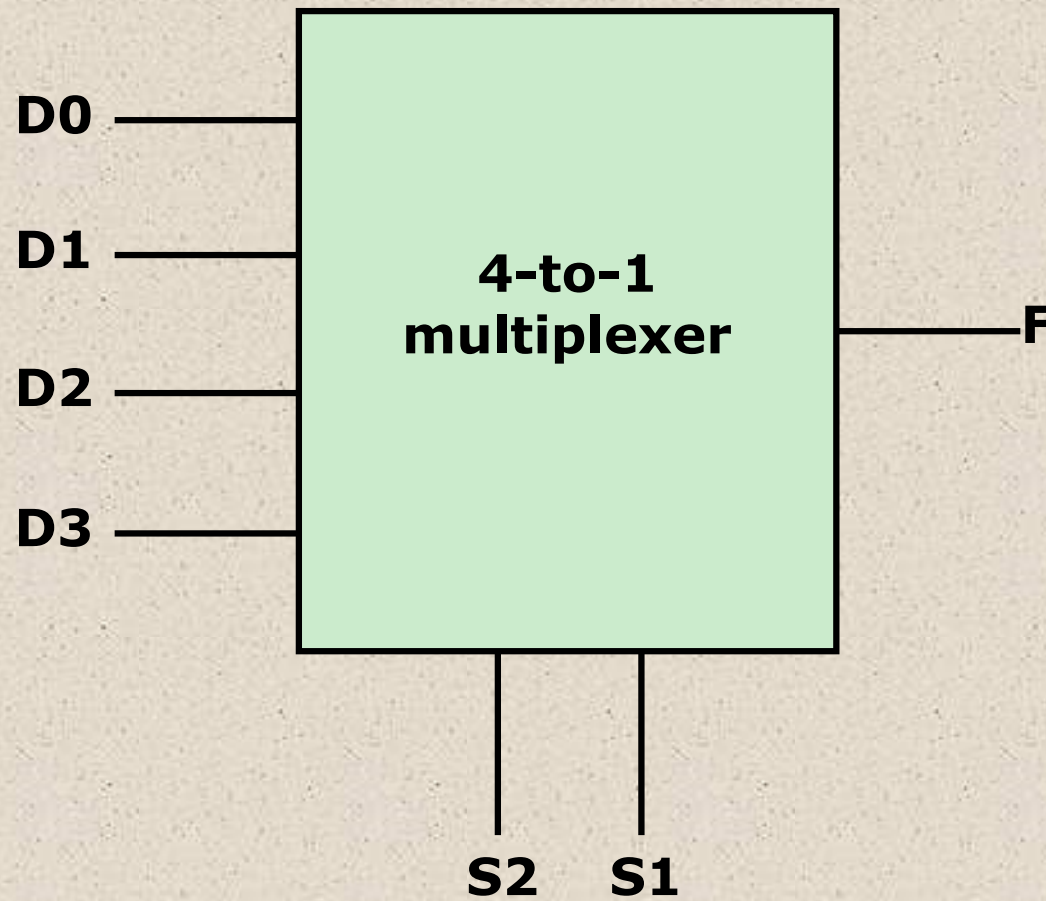


Figure 11.12 4-to-1 Multiplexer Representation



Table 11.7
4-to-1 Multiplexer Truth Table

S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3

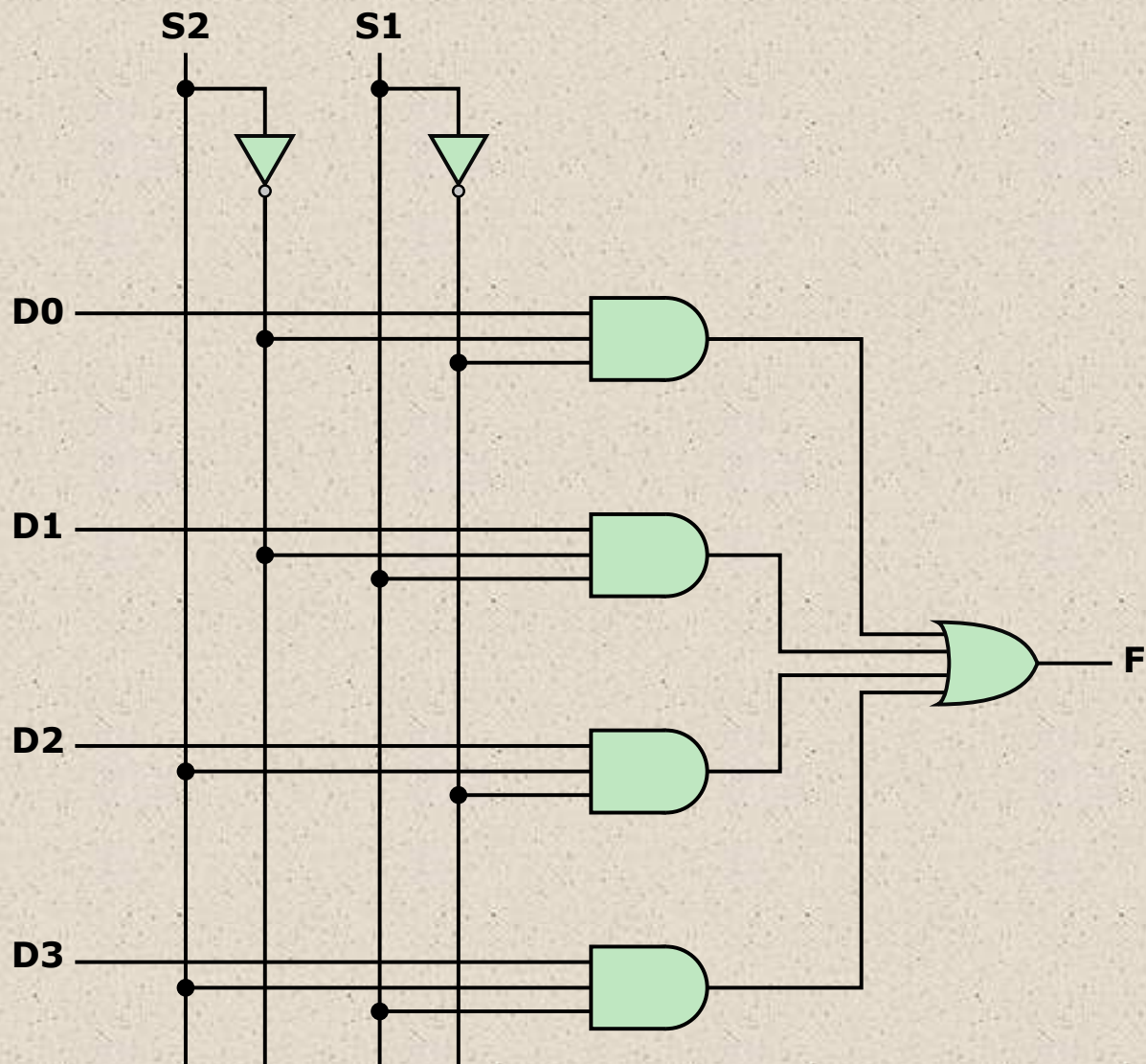


Figure 11.13 Multiplexer Implementation

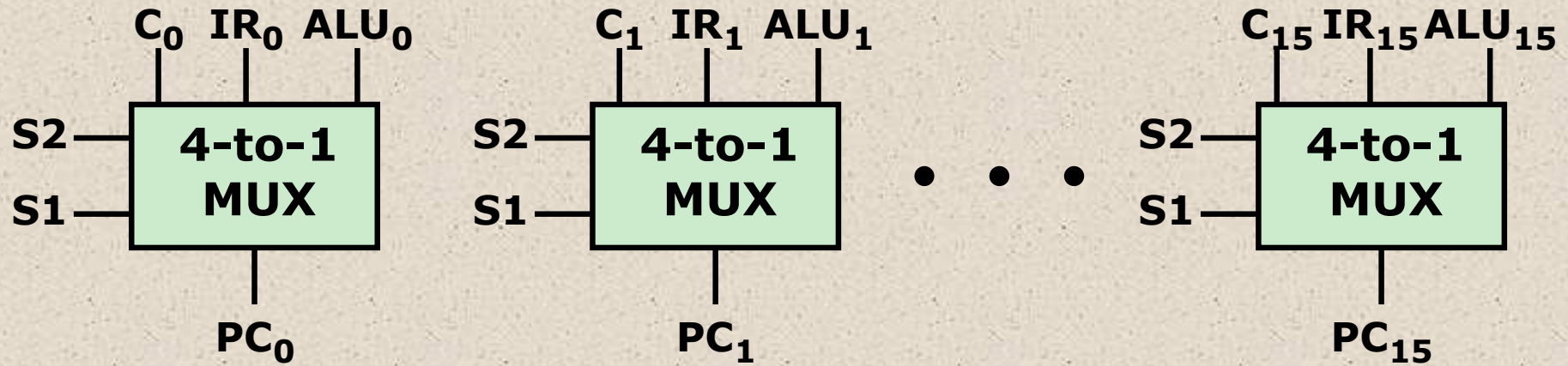


Figure 11.14 Multiplexer Input to Program Counter

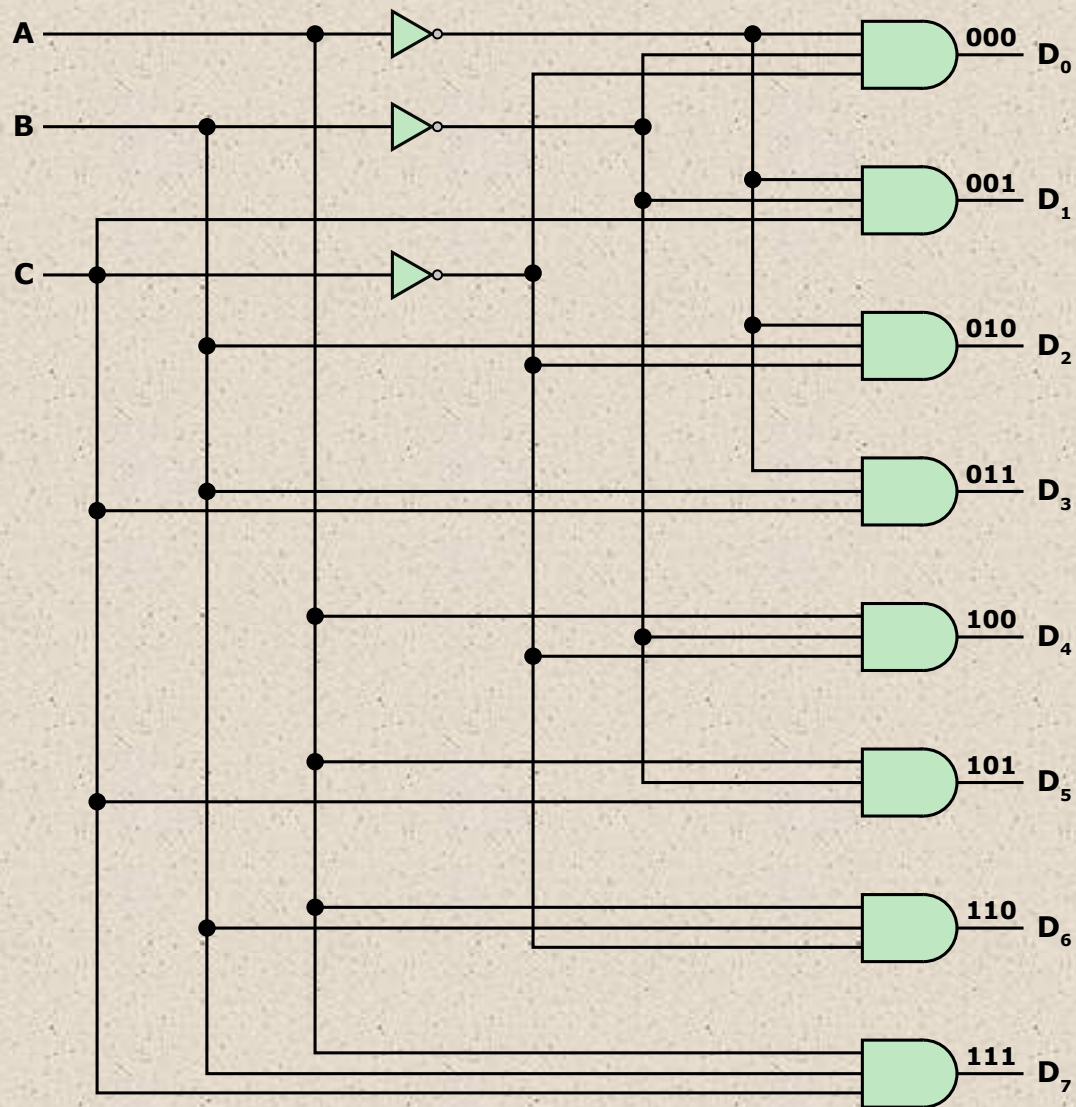


Figure 11.15 Decoder with 3 Inputs and $2^3 = 8$ Outputs

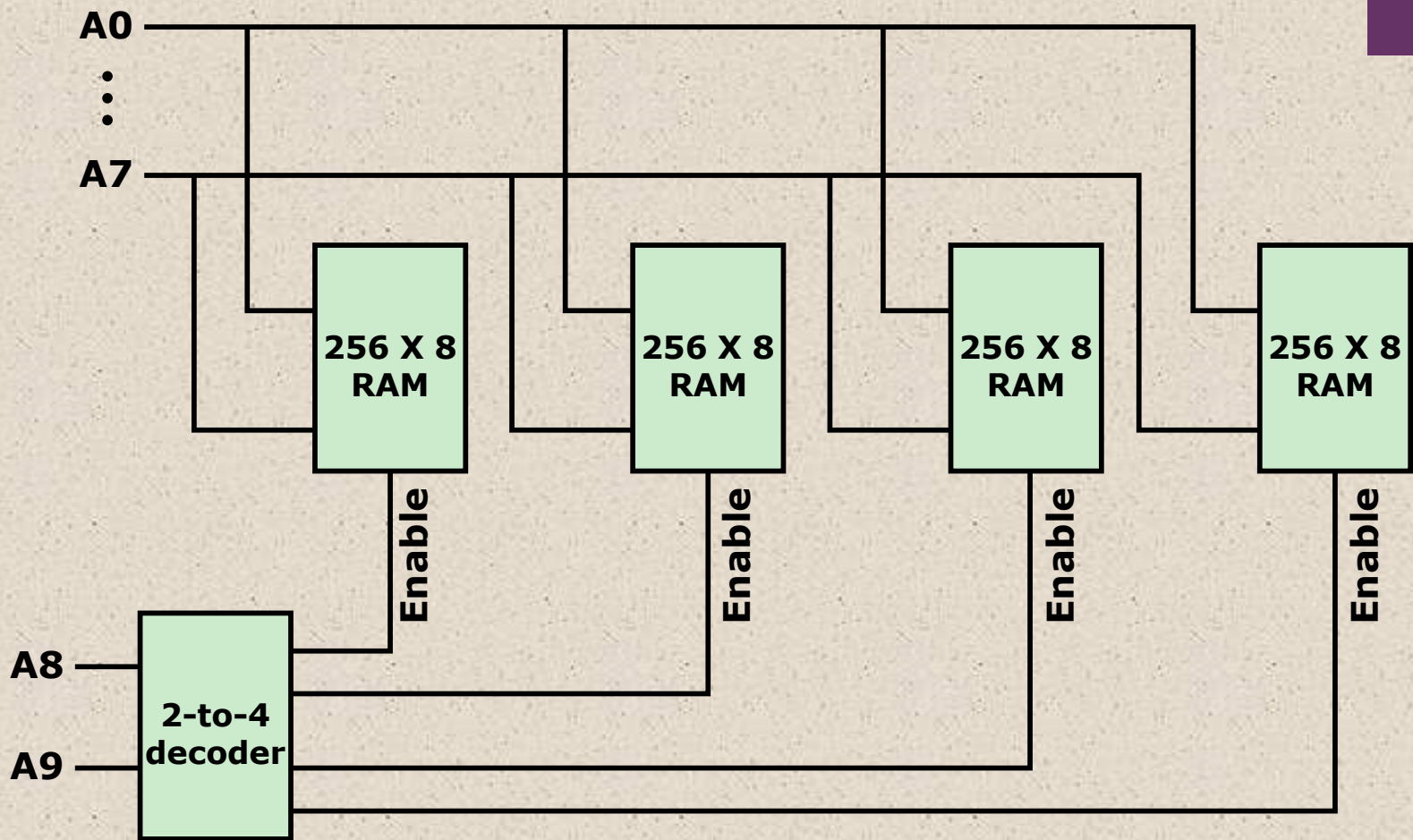


Figure 11.16 Address Decoding

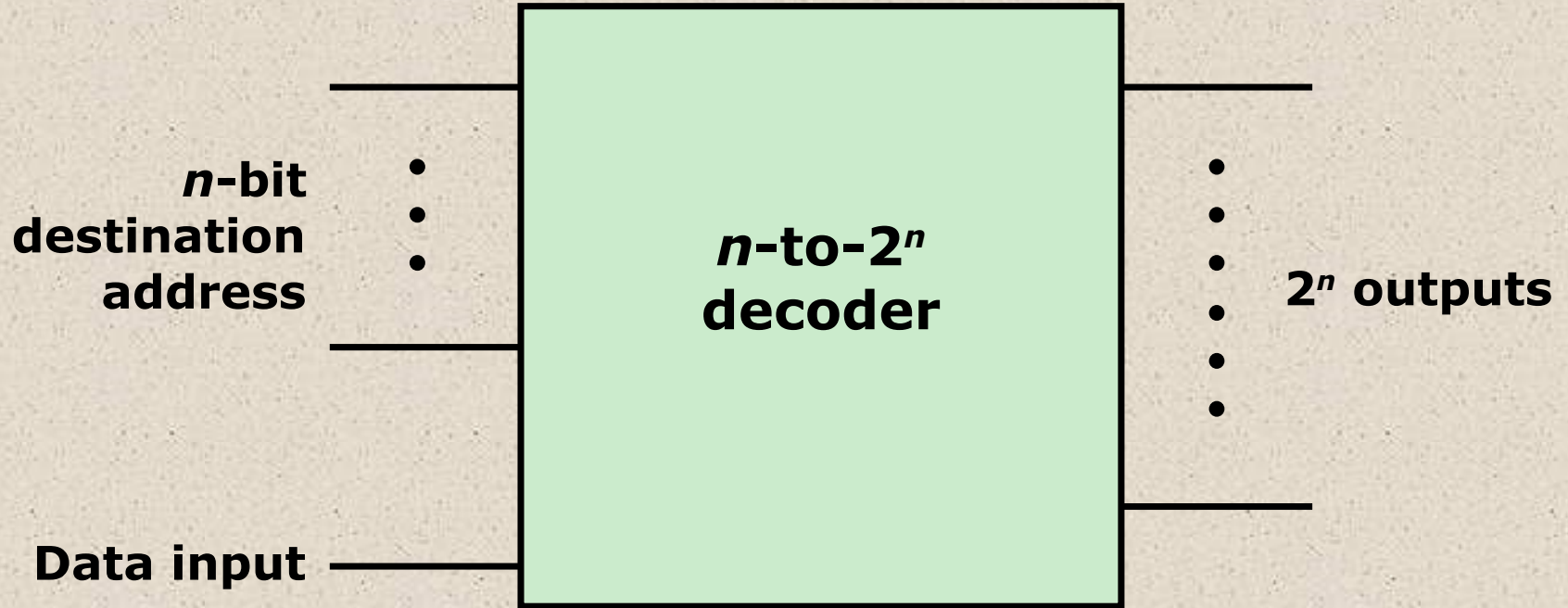


Figure 11.17 Implementation of a Demultiplexer Using a Decoder

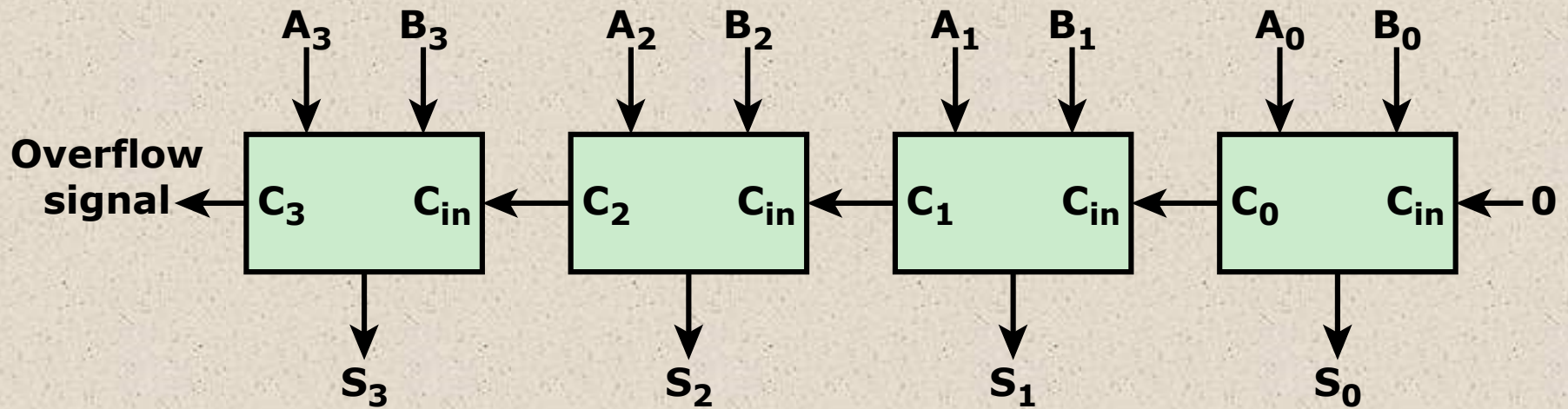


Figure 11.19 4-Bit Adder

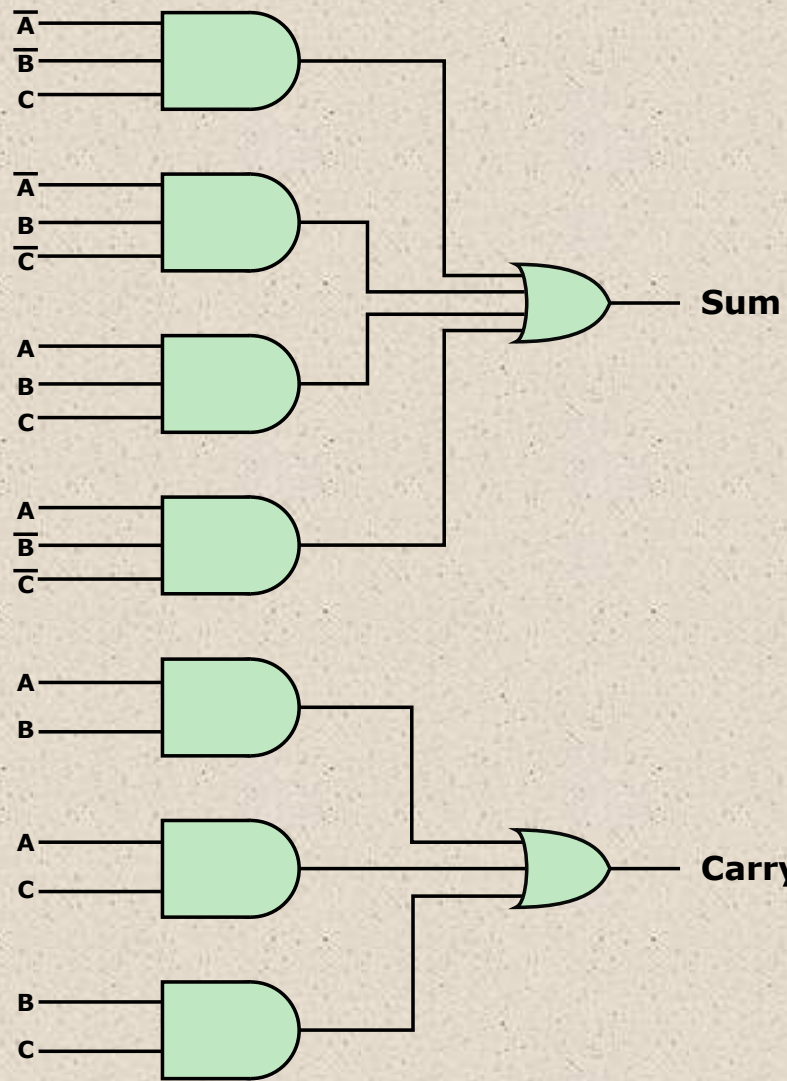


Figure 11.20 Implementation of an Adder

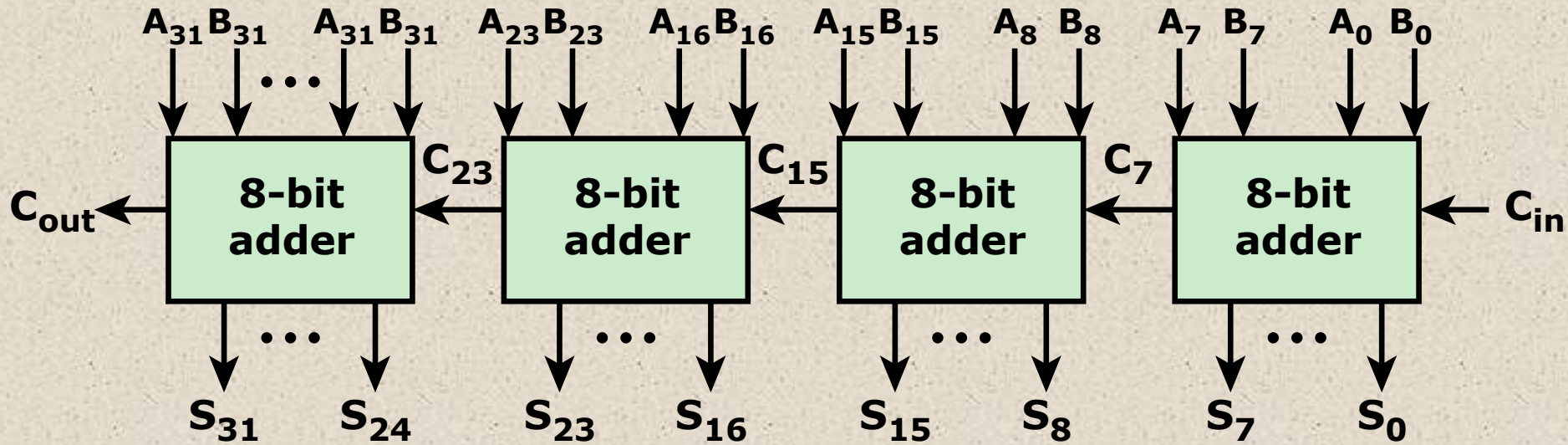
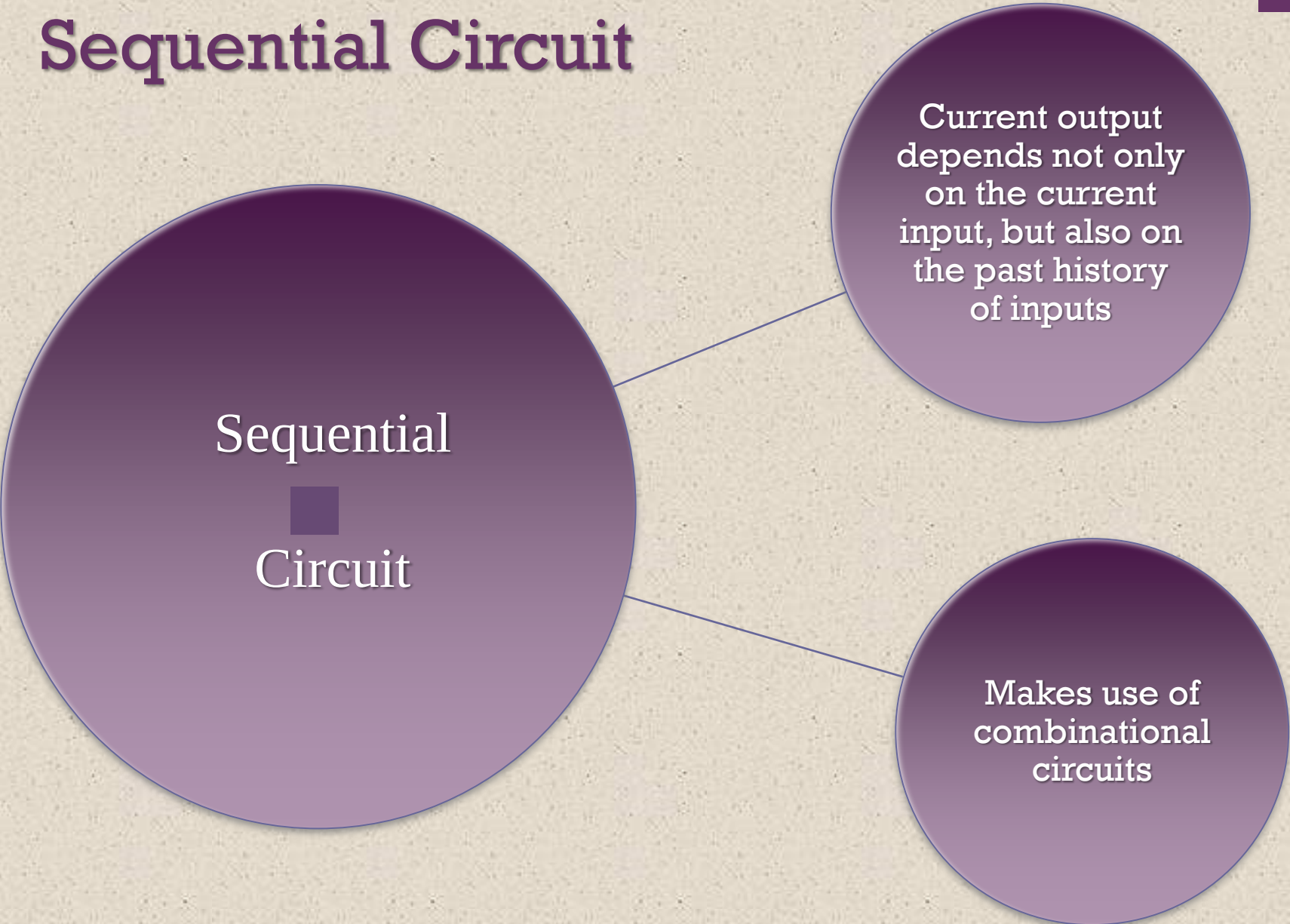


Figure 11.21 Construction of a 32-Bit Adder Using 8-Bit Adders

Sequential Circuit

Sequential
Circuit



```
graph TD; A((Sequential Circuit)) --- B((Current output depends not only on the current input, but also on the past history of inputs)); A --- C((Makes use of combinational circuits))
```

Current output depends not only on the current input, but also on the past history of inputs

Makes use of combinational circuits



Flip-Flops



- Simplest form of sequential circuit
- There are a variety of flip-flops, all of which share two properties:
 1. The flip-flop is a bistable device. It exists in one of two states and, in the absence of input, remains in that state. Thus, the flip-flop can function as a 1-bit memory.
 2. The flip-flop has two outputs, which are always the complements of each other.

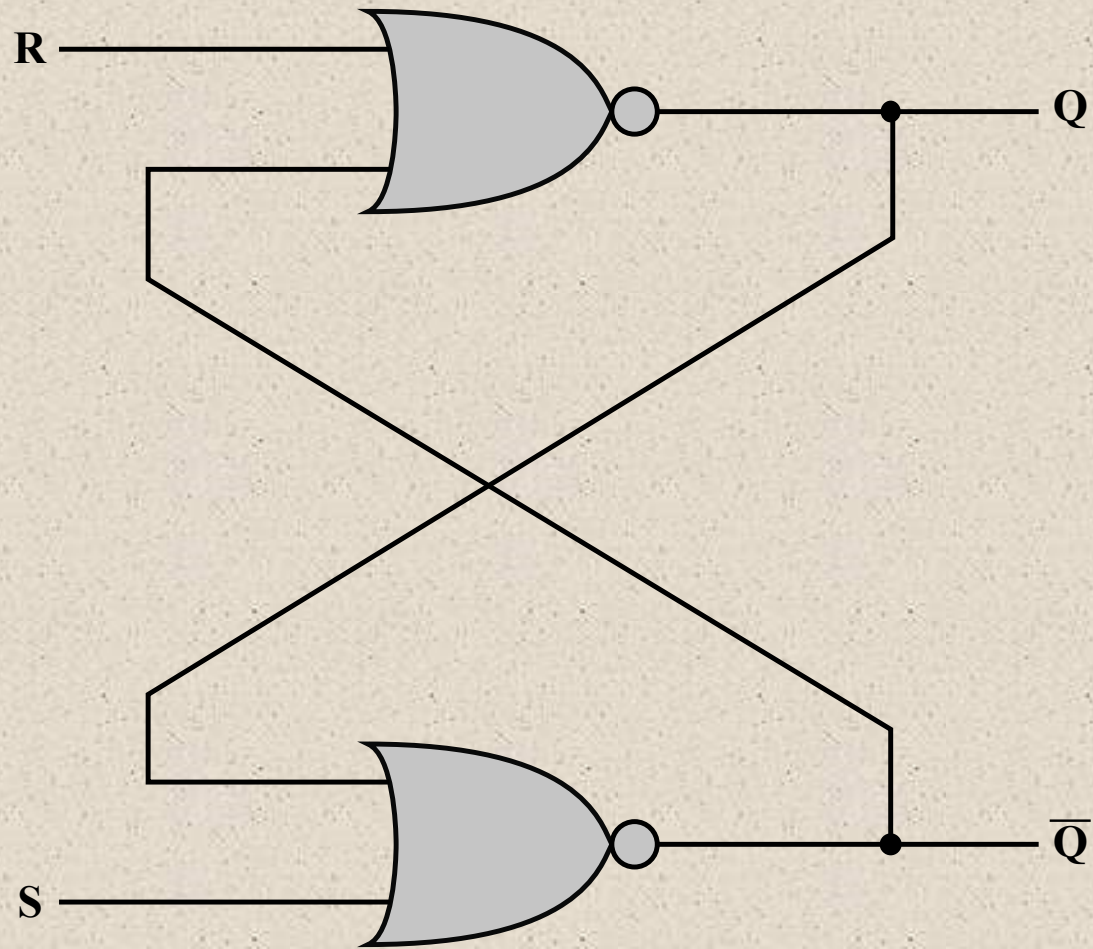


Figure 11.22 The S-R Latch Implemented with NOR Gates

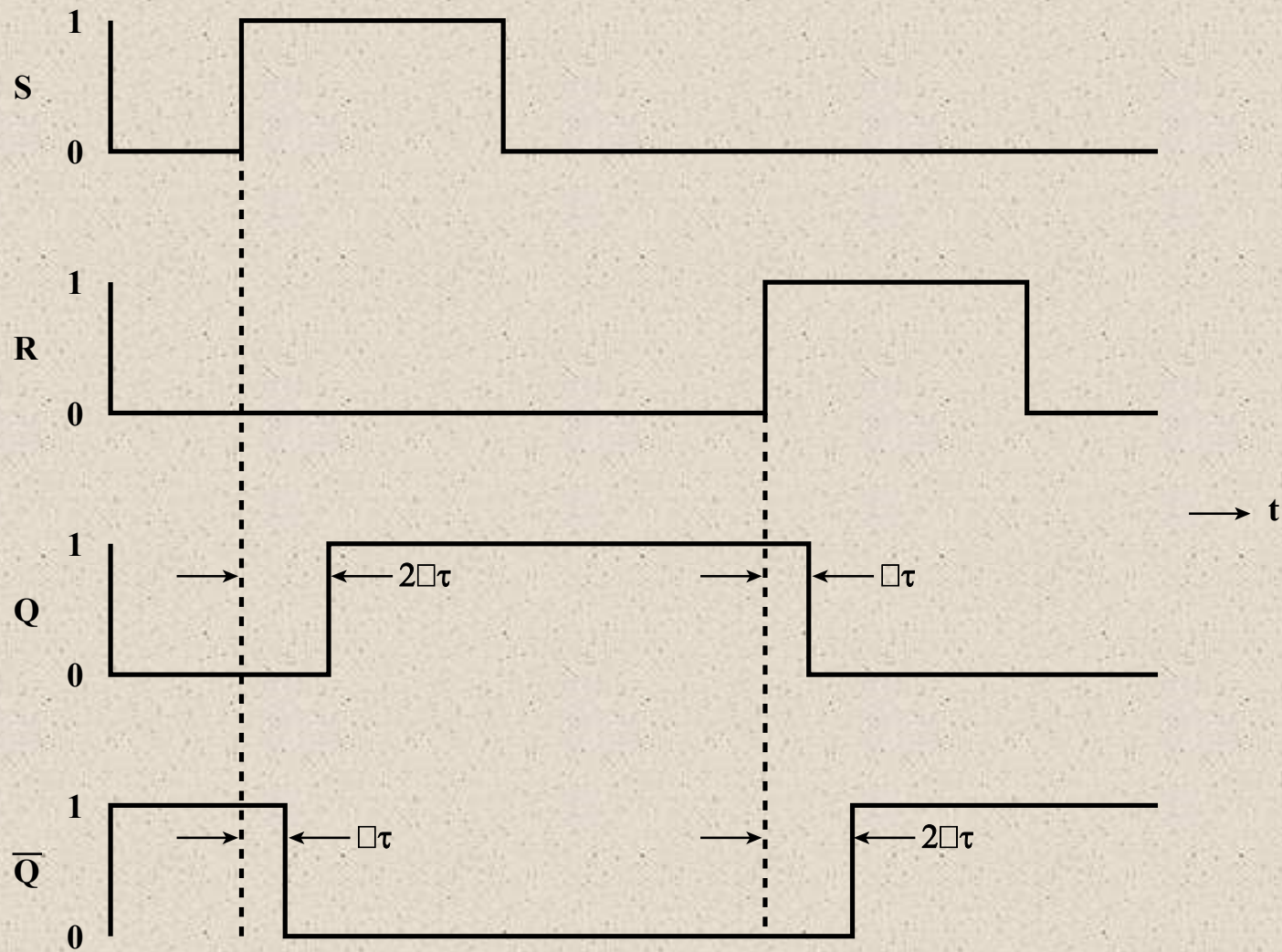


Figure 11.23 NOR S-R Latch Timing Diagram

Table 11.10 The S-R Latch



(a) Characteristic Table

Current Inputs SR	Current State Q_n	Next State Q_{n+1}
00	0	0
00	1	1
01	0	0
01	1	0
10	0	1
10	1	1
11	0	—
11	1	—

(b) Simplified Characteristic Table

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	—

(c) Response to Series of Inputs

t	0	1	2	3	4	5	6	7	8	9
S	1	0	0	0	0	0	0	0	1	0
R	0	0	0	1	0	0	1	0	0	0
Q_{n+1}	1	1	1	0	0	0	0	0	1	1

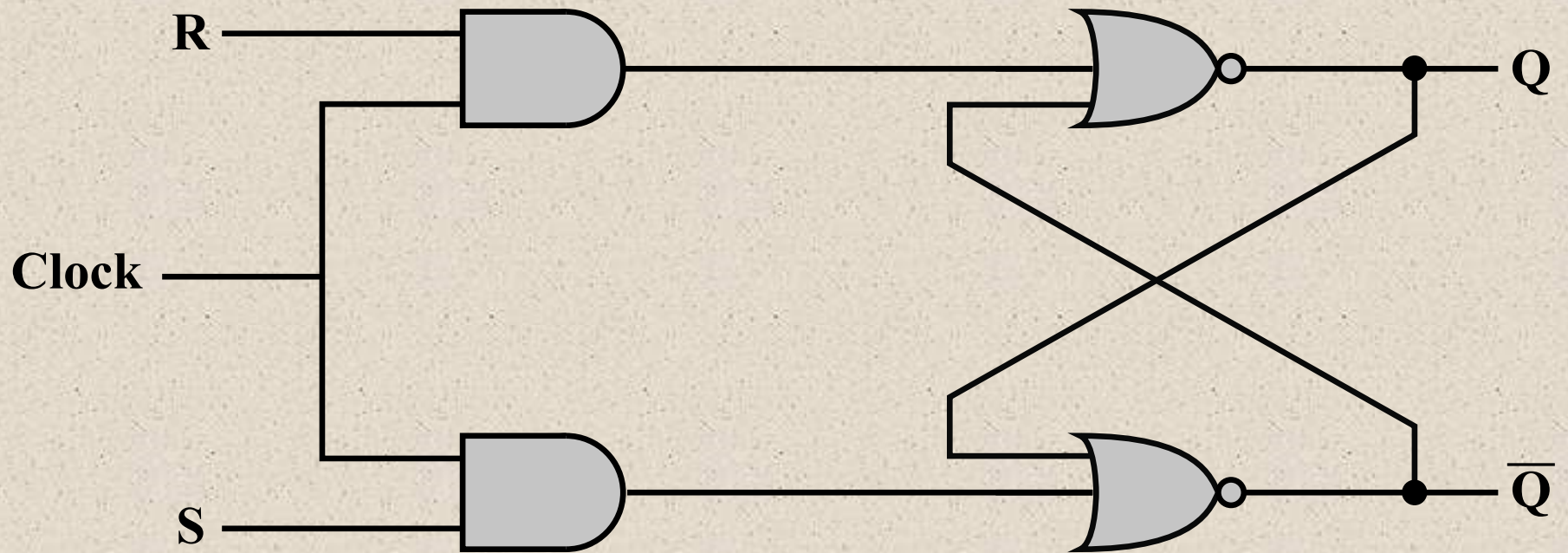


Figure 11.24 Clocked S-R Flip Flop

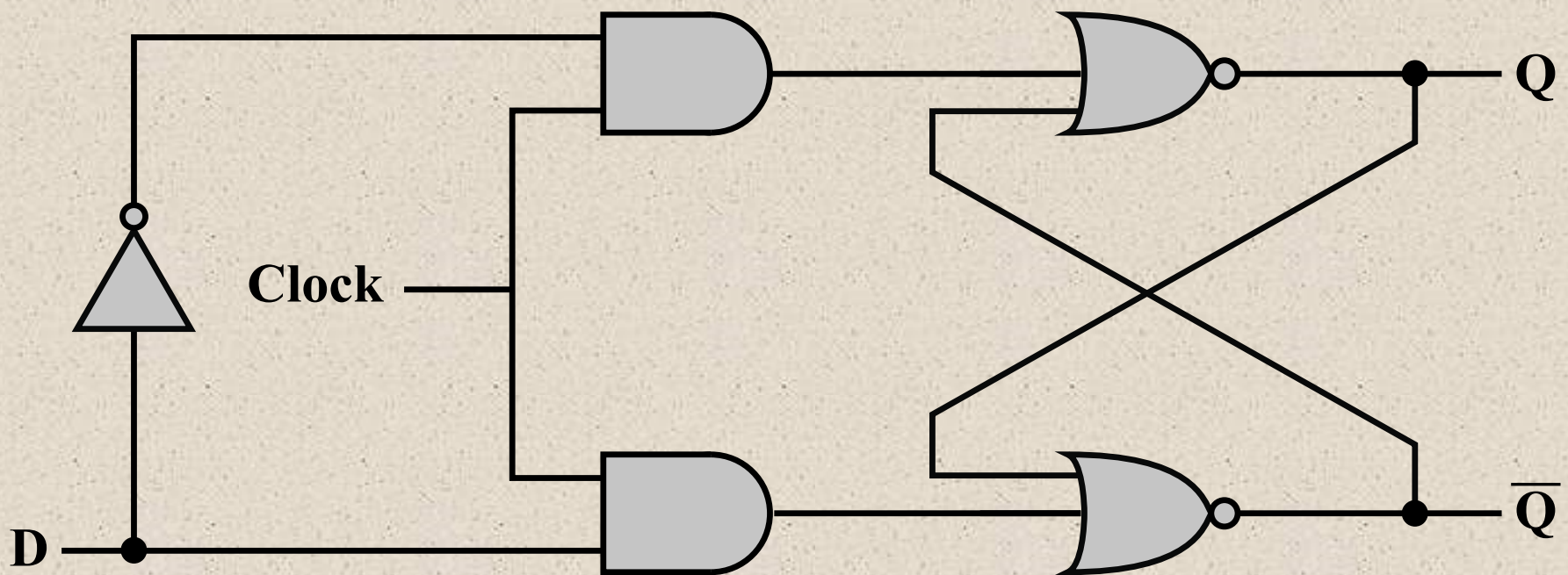


Figure 11.25 D Flip Flop

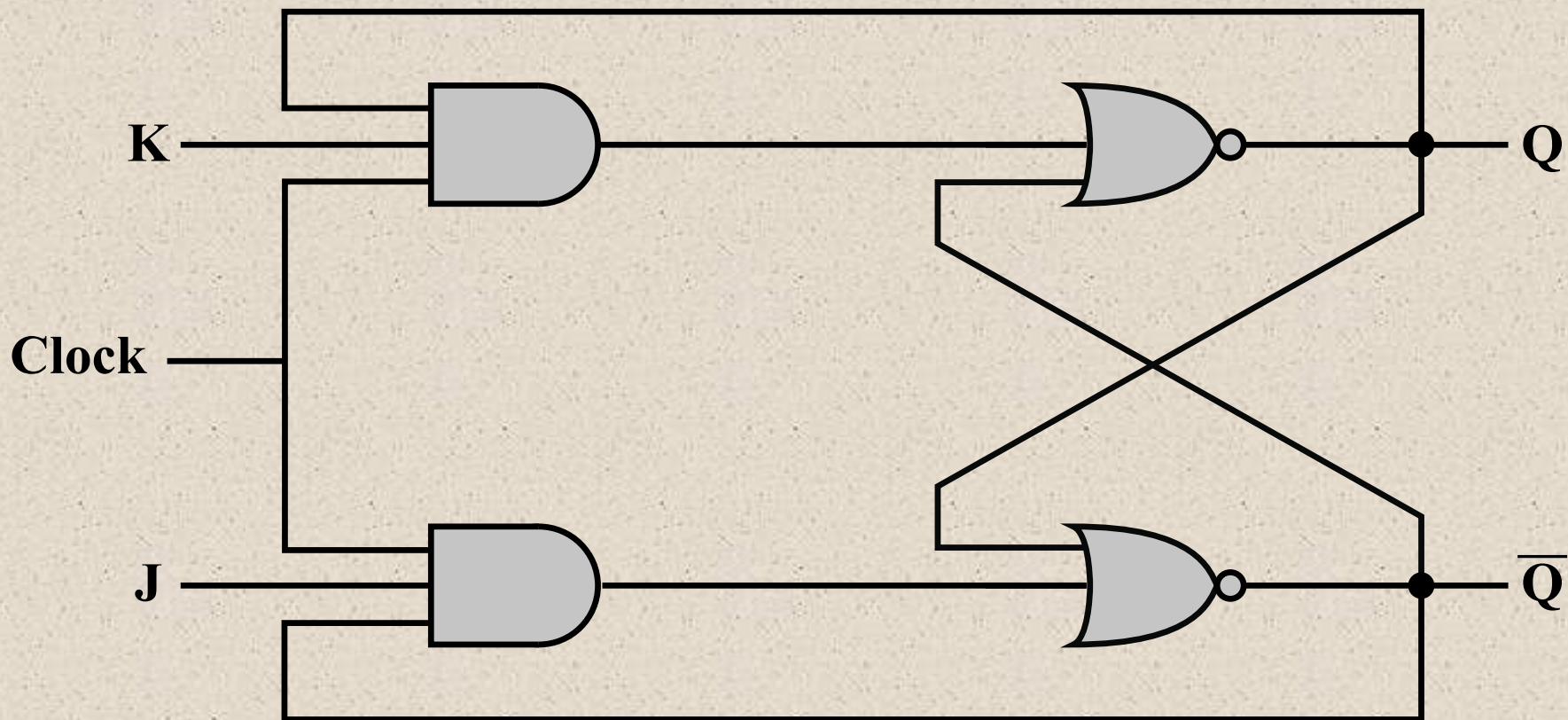


Figure 11.26 J-K Flip Flop

Name	Graphical Symbol	Truth Table															
S-R		<table><tr><th>S</th><th>R</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td><td>Q_n</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>—</td></tr></table>	S	R	Q_{n+1}	0	0	Q_n	0	1	0	1	0	1	1	1	—
S	R	Q_{n+1}															
0	0	Q_n															
0	1	0															
1	0	1															
1	1	—															
J-K		<table><tr><th>J</th><th>K</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td><td>Q_n</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>$\overline{Q_n}$</td></tr></table>	J	K	Q_{n+1}	0	0	Q_n	0	1	0	1	0	1	1	1	$\overline{Q_n}$
J	K	Q_{n+1}															
0	0	Q_n															
0	1	0															
1	0	1															
1	1	$\overline{Q_n}$															
D		<table><tr><th>D</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q_{n+1}	0	0	1	1									
D	Q_{n+1}																
0	0																
1	1																

Figure 11.27 Basic Flip-Flops



REGISTERS and COUNTERS

Registers

- Register: a set of n flip-flops used to store n bits of information
- A common clock is used for all the flip-flops
- A register that provides the ability to shift its contents is called a shift register
- To implement a shift register, it is necessary to use edge-triggered or master-slave flip-flops



REGISTERS



- SISO (Serial in Serial Out)
- SIPO (Serial in Parallel Out)
- **PIPO (Parallel in Parallel Out)**
- PISO (Parallel in Serial Out)



Counter



- A register whose value is easily incremented by 1 modulo the capacity of the register
- After the maximum value is achieved the next increment sets the counter value to 0
- An example of a counter in the CPU is the program counter
- Can be designated as:
 - Asynchronous/Ripple Counter
 - Relatively slow because the output of one flip-flop triggers a change in the status of the next flip-flop
 - Synchronous
 - All of the flip-flops change state at the same time
 - Because it is faster it is the kind used in CPUs



Triggering Methods

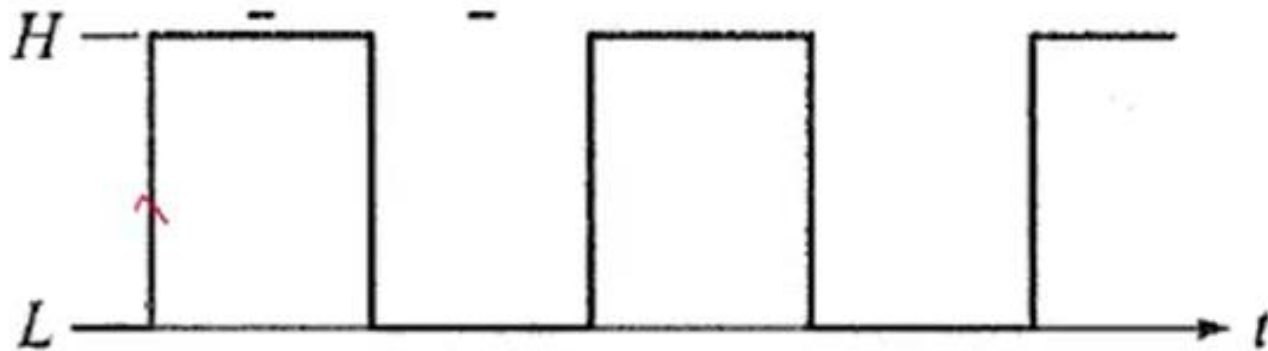
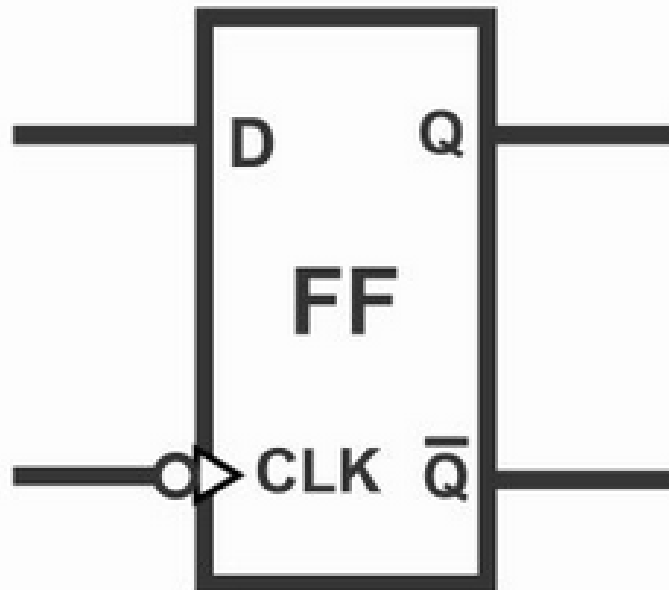
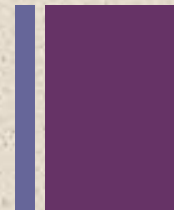


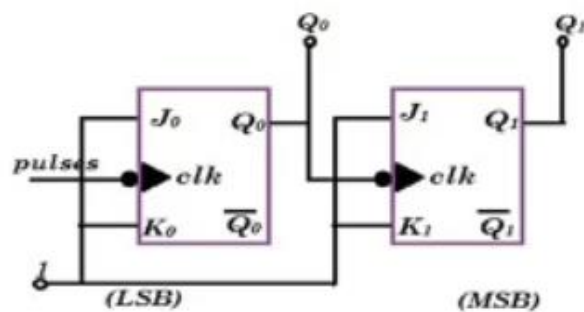


Figure to present Negative edge Triggering

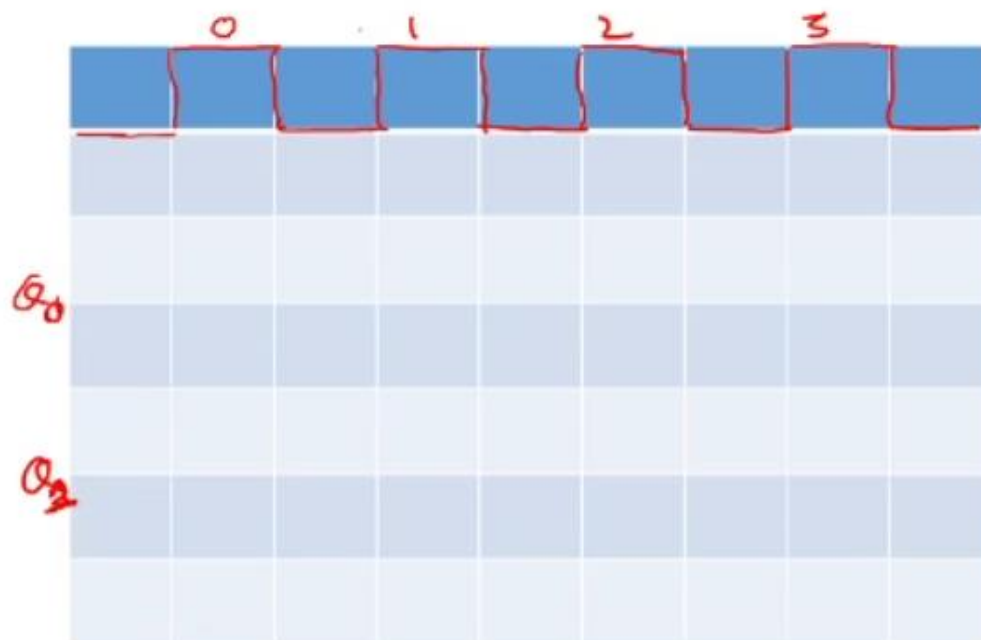




Ripple Counter : 2-bit Up counter



CLK	Q1	Q0





2-Bit Counter



- Decide about number and type of flip-flops
- Draw the excitation of the flip-flop
- Draw the State diagram
- Obtain the circuit excitation table
- Obtain the simplified Boolean function using K-Map
- Draw the Logic Diagram



Decide about number and type of flip-flops



- Type of Flip-Flop: JK
- No of flip flops:2

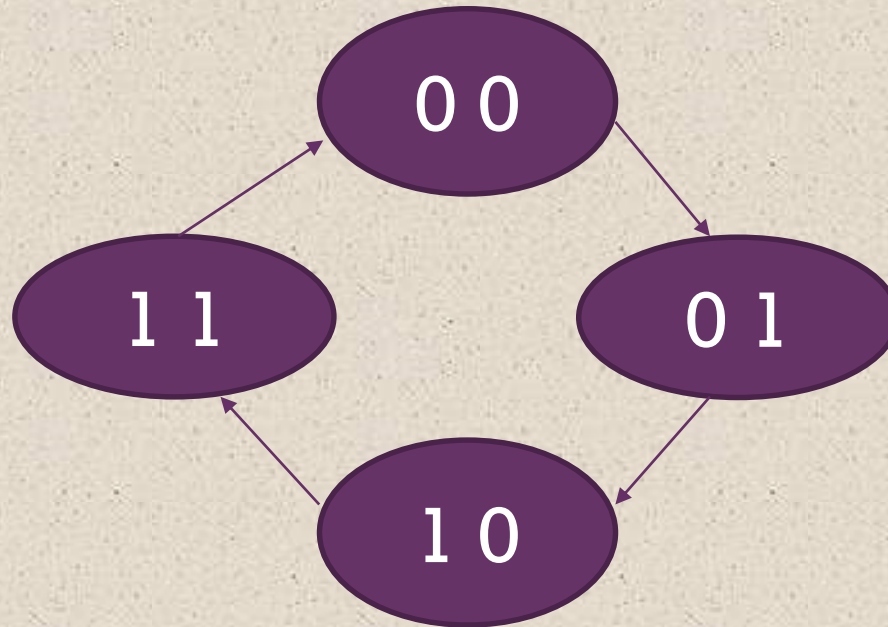
+

Draw the excitation of the flip-flop

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

+

Draw the State diagram



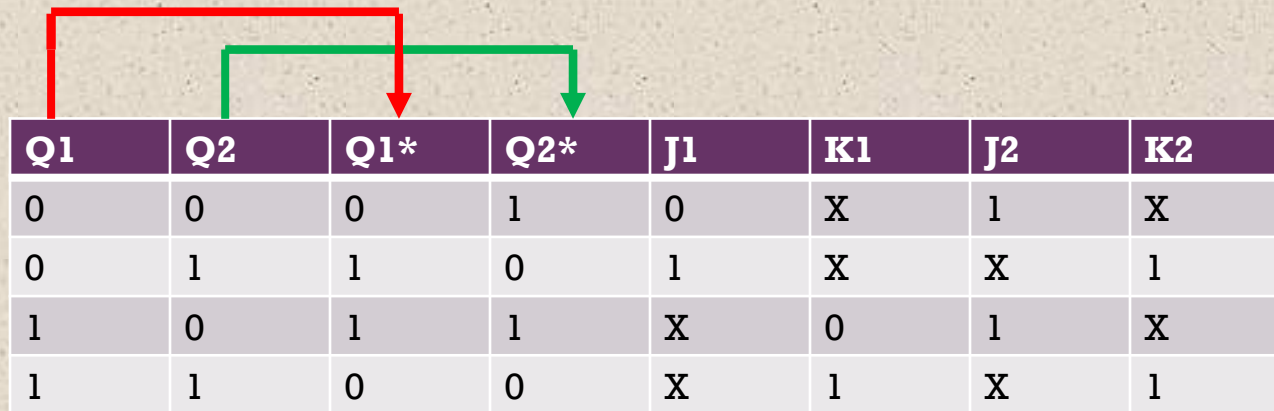
+

Obtain the circuit excitation table

Q1	Q2	Q1*	Q2*	J1	K1	J2	K2
0	0	0	1				
0	1	1	0				
1	0	1	1				
1	1	0	0				

+

Obtain the simplified Boolean function using K-Map

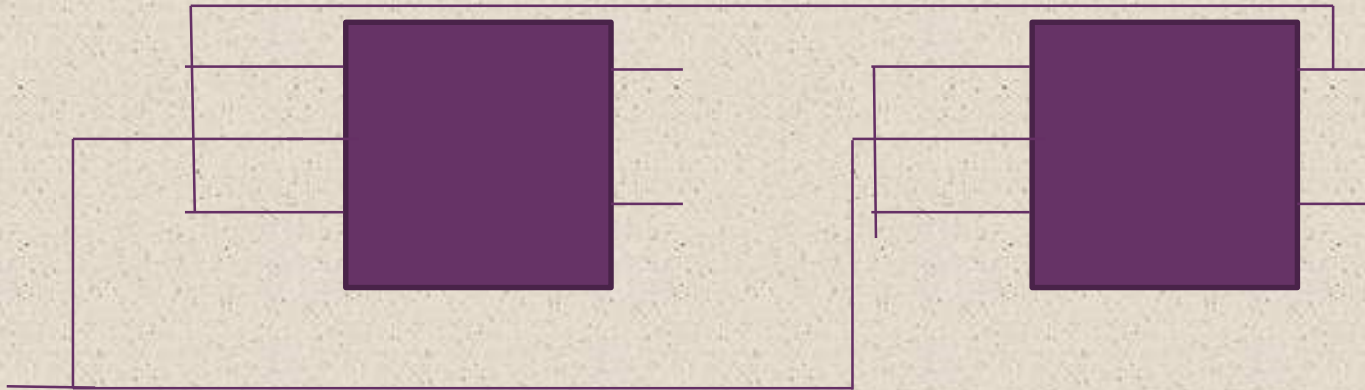


A Karnaugh map for variables Q1 and Q2. The map is a 2x2 grid with columns labeled Q1 and Q2, and rows labeled Q1* and Q2*. The cells contain values for J1 and K1. A red arrow groups the top-left and top-right cells (J1=0, K1=X). A green arrow groups the top-right and bottom-right cells (J1=1, K1=1).

Q1	Q2	Q1*	Q2*	J1	K1	J2	K2
0	0	0	1	0	X	1	X
0	1	1	0	1	X	X	1
1	0	1	1	X	0	1	X
1	1	0	0	X	1	X	1

+

Draw the Logic Diagram





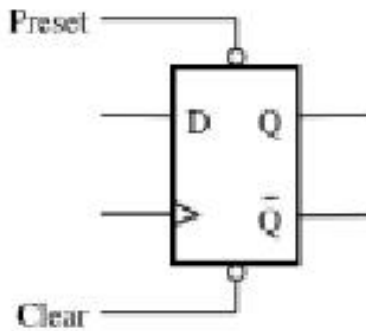
Shift Registers



Clear: asynchronous, synchronous

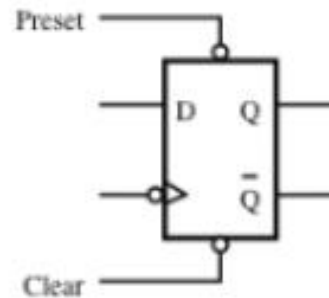
Asynchronous clear: flip-flops are cleared without regard to clock signal

Synchronous clear: flip-flops are clear with the clock signal



(b) Graphical symbol

Positive-edge-triggered D flip-flop with *Clear* and *Preset*.



(b) Graphical symbol

Master-slave D flip-flop with *Clear* and *Preset*.

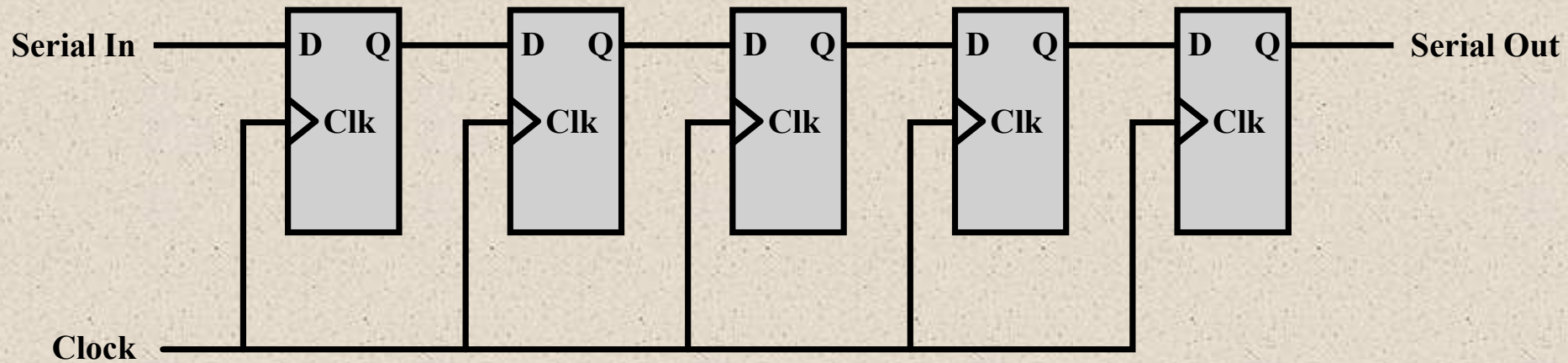
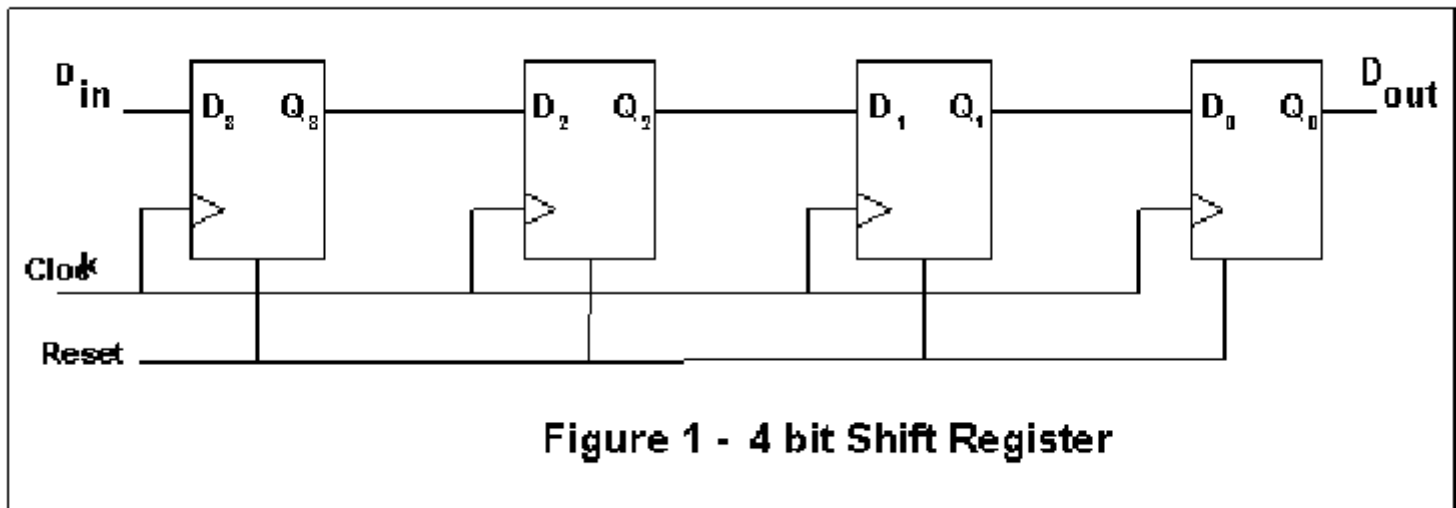


Figure 11.29 5-Bit Shift Register



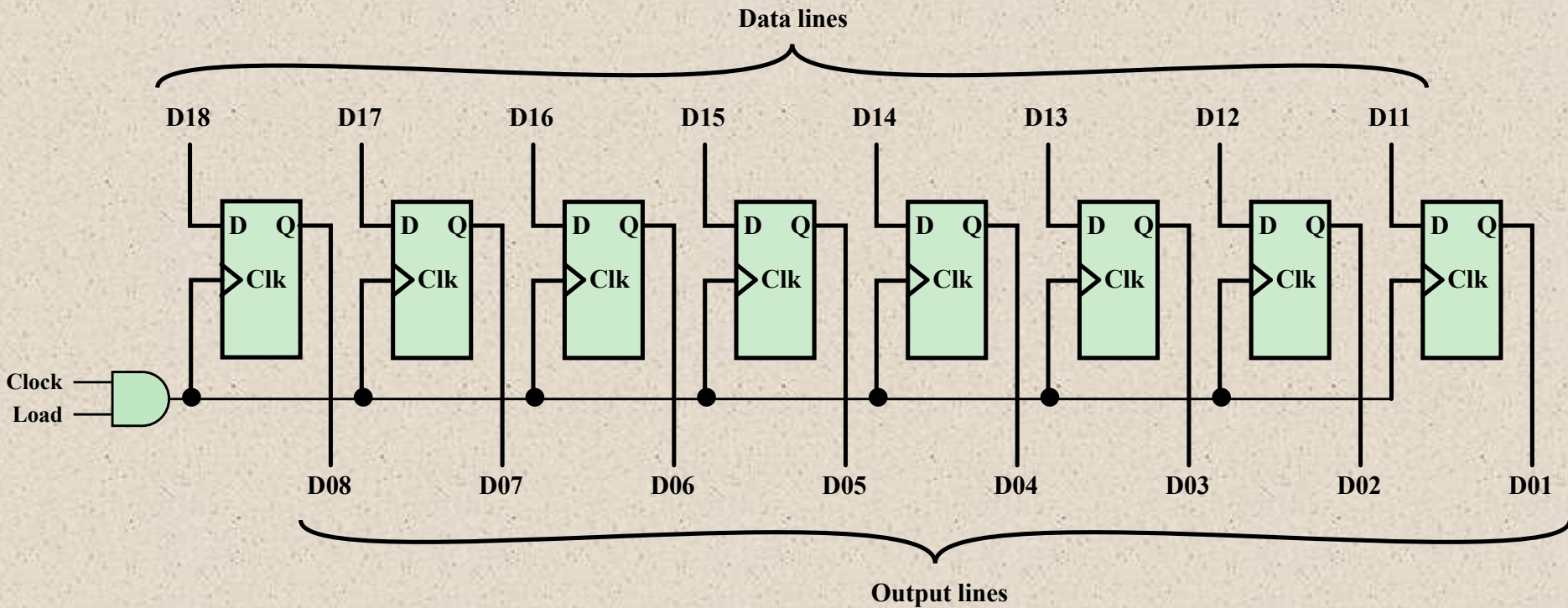
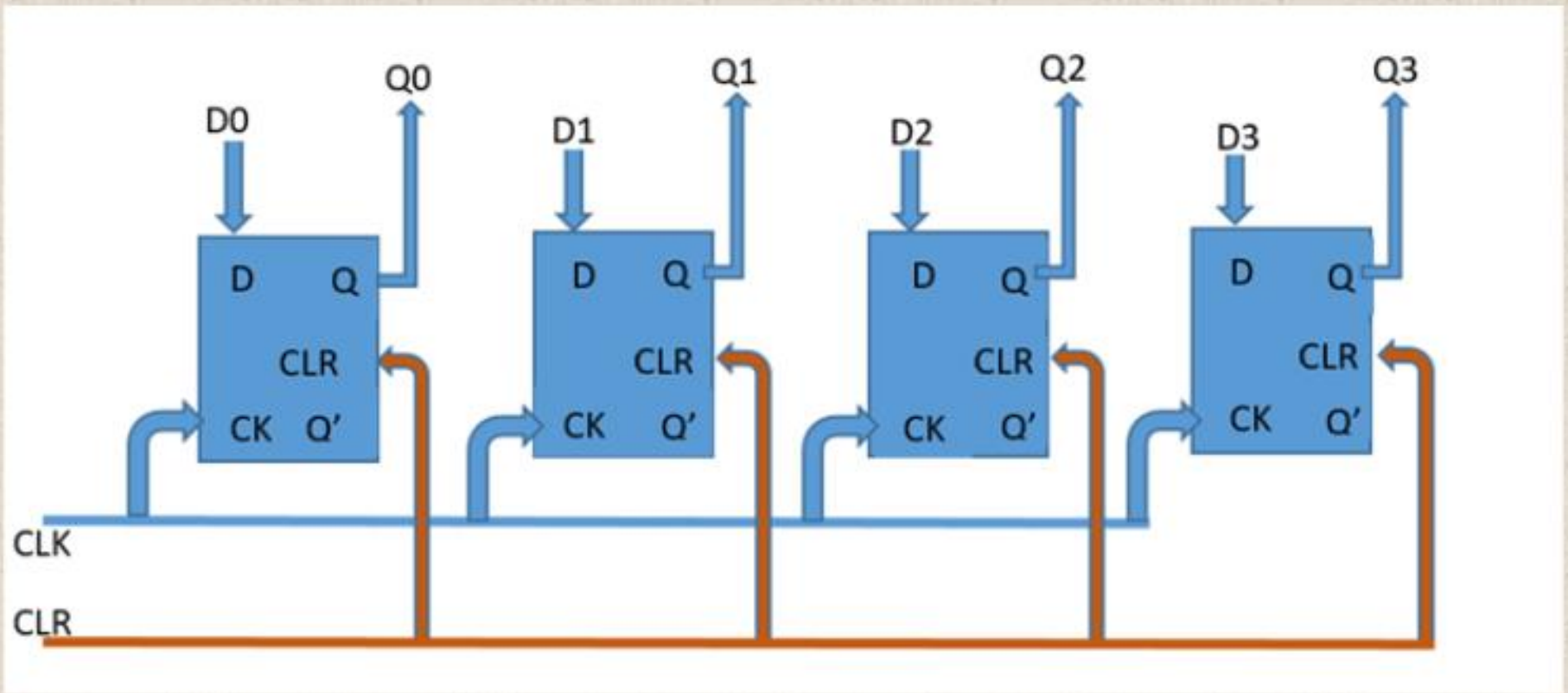


Figure 11.28 8-Bit Parallel Register





Jai Sairam

Programmable Logic Device (PLD)

A general term that refers to any type of integrated circuit used for implementing digital hardware, where the chip can be configured by the end user to realize different designs. Programming of such a device often involves placing the chip into a special programming unit, but some chips can also be configured “in-system”. Also referred to as a field-programmable device (FPD).

Programmable Logic Array (PLA)

A relatively small PLD that contains two levels of logic, an AND-plane and an OR-plane, where both levels are programmable.

Programmable Array Logic (PAL)

A relatively small PLD that has a programmable AND-plane followed by a fixed OR-plane.

Simple PLD (SPLD)

A PLA or PAL.

Complex PLD (CPLD)

A more complex PLD that consists of an arrangement of multiple SPLD-like blocks on a single chip.

Field-Programmable Gate Array (FPGA)

+ A PLD featuring a general structure that allows very high logic capacity. Whereas CPLDs feature logic resources with a wide number of inputs (AND planes), FPGAs offer more narrow logic resources. FPGAs also offer a higher ratio of flip-flops to logic resources than do CPLDs.

Logic Block

A relatively small circuit block that is replicated in an array in an FPD. When a circuit is implemented in an FPD, it is first decomposed into smaller sub-circuits that can each be mapped into a logic block. The term logic block is mostly used in the context of FPGAs, but it could also refer to a block of circuitry in a CPLD.

Table

11.11

PLD

Terminology

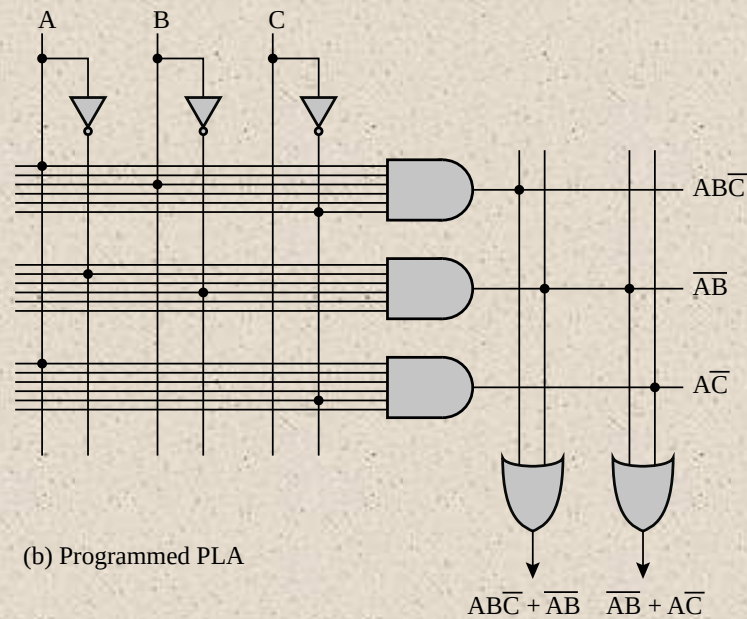
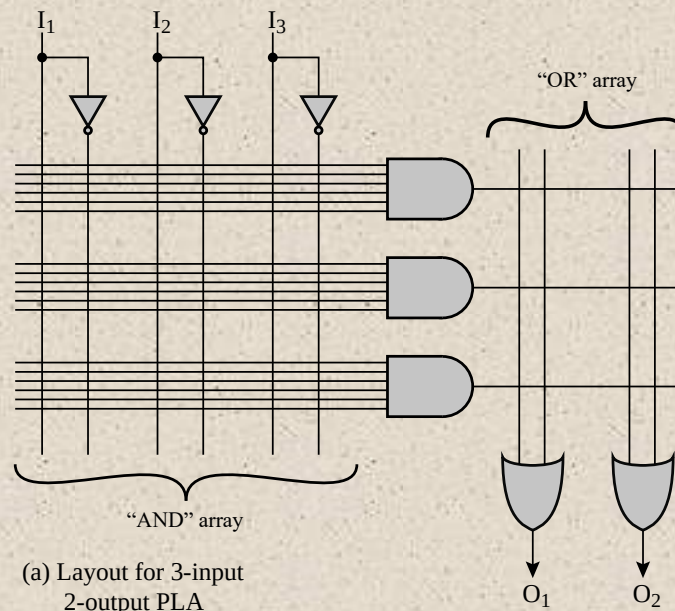
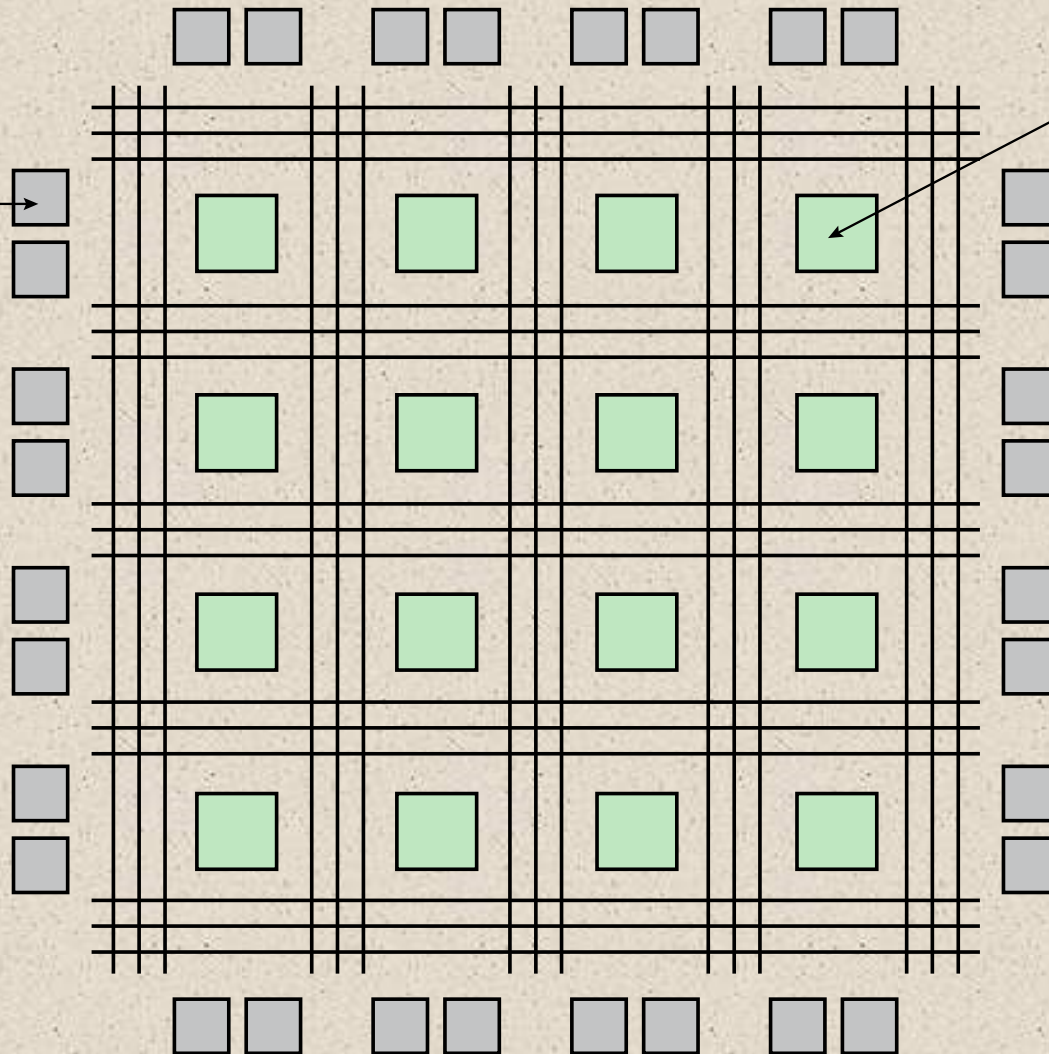
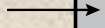


Figure 11.32 An Example of a Programmable Logic Array

**I/O
block**



**Logic
block**



Figure 11.33 Structure of an FPGA

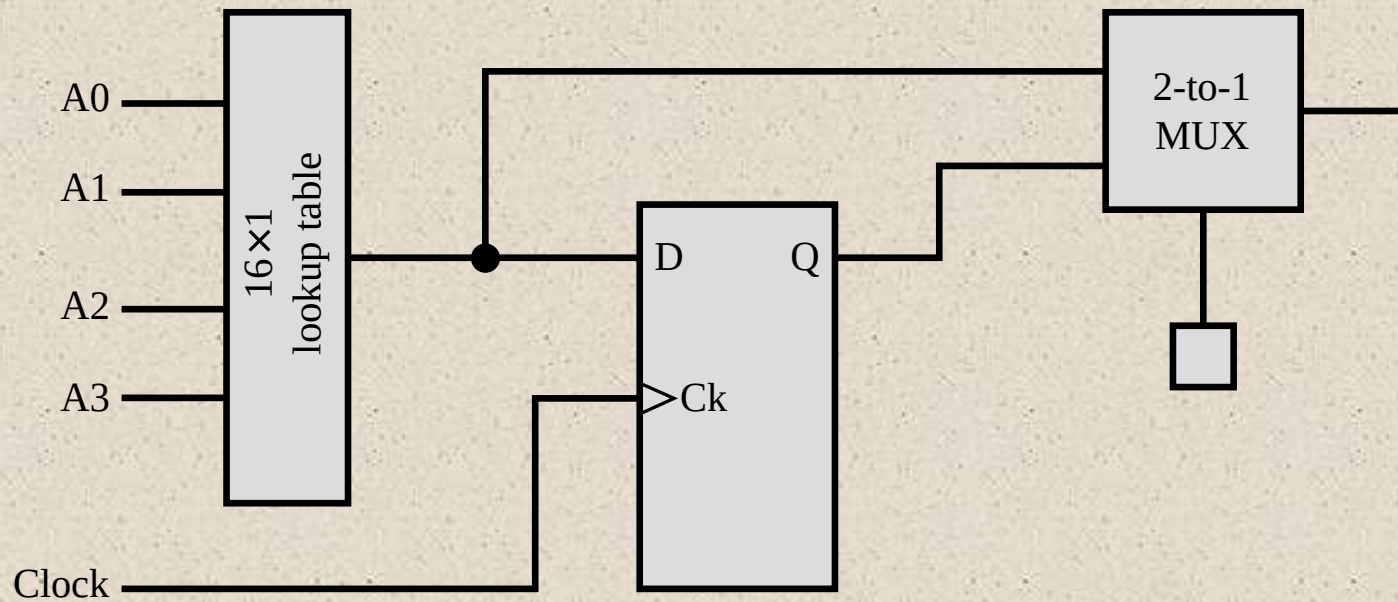


Figure 11.34 A Simple FPGA Logic Block

+ Summary

Chapter 11

Digital Logic

- Boolean Algebra
- Gates
- Combinational Circuits
 - Implementation of Boolean Functions
 - Multiplexers
 - Decoders
 - Read-Only-Memory
 - Adders
- Sequential Circuits
 - Flip-Flops
 - Registers
 - Counters
- Programmable Logic Devices
 - Programmable Logic Array
 - Field-Programmable Gate Array